

Scale-Invariant Feature Transform (SIFT)

Prepared By
Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg

Instructor:

Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg



جامعة العلوم والتكنولوجيا
University of Science and Technology

Introduction

- The **Scale-Invariant Feature Transform (SIFT)** is a feature detection algorithm introduced by David G. Lowe in **1999** and refined in **2004**.
- It has been widely used in computer vision tasks due to its robustness in **detecting key features in images** under various conditions.

SIFT Properties

Scale Invariance:

Detects features at different scales of the image, making it **robust to changes in size**.

Rotation Invariance:

Features are detected **independently of the image orientation**.

Affine Transform Invariance:

Handles distortions such as scaling and rotation simultaneously.

3D Viewpoint Invariance:

The features remain robust even with **changes in the 3D viewpoint**.

Illumination Invariance:

Changes in lighting conditions do not significantly impact feature detection.

Occlusion Invariance:

Handles partial occlusions of objects within the scene.

Demo of SIFT Feature



90 Degree



45 Degree



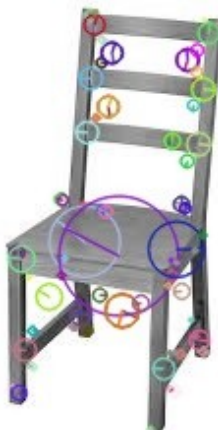
Same Points



Demo of SIFT Feature



Connecting Keypoints pairs of both images if their descriptors are matching



Rotated and Scaled
(Still same points)



Applications Utilizing SIFT

Image Matching:

SIFT is highly effective in finding corresponding features between different images.

Image alignment and stitching:

Used in Panorama Creation

Object or Scene Recognition:

Useful for recognizing objects or scenes based on unique feature points.

3D Structure Recovery:

SIFT features can help solve for 3D structure from multiple images.

Stereo Correspondence:

Assists in matching points in stereo image pairs.

Motion Tracking:

Used for tracking objects through video frames.

Main Idea of SIFT

7

Two Main steps:

- [1] **Detect** “Keypoints” of the Image
- [2] **Generate** “Descriptor” for each keypoint

What is “Key Point”

KEYpoint is a point in the image

*Corners and points on Edges are good “Points” but not really interesting enough points for a lot of applications,
we need things that are a little bit more descriptive.*

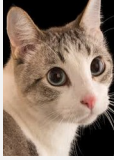
blob, which has some local appearance within the image is potentially a very good interesting KEY point.

Main Idea of SIFT

8

1

Generate Octaves of the input Image



2

Keypoint Detection (at different Scales)



3

Refinement of Detected Keypoints



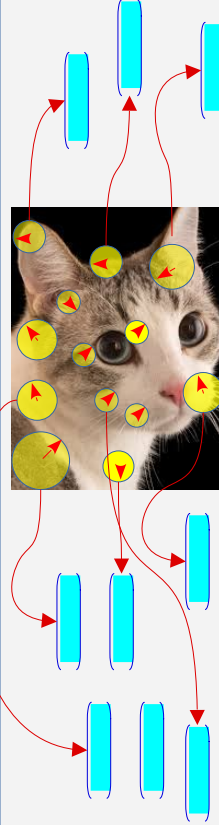
4

Keypoint
[Position – Scale – Orientation]



5

Build Descriptor for each Keypoint



Detection of key points (Blobs)

Main Idea:

Use derivatives of images to actually detect blobs (similar to detection of Edges)

Derivatives should handle not only the **position** of the blob but also the **magnification** of the blob

Remember: Object can appear at many different magnifications depending on its distance from the camera.

Conclusion: We have to be able to deal with scale.

Note:

Lot of Blobs will be detected **using Gradient**.
We have to select the most suitable blobs

Steps to detect key points (Blobs)

[1] Constructing a Scale Space

To make sure that features are scale-independent

[2] Key point Localization: Identifying the suitable features or key points

[3] Orientation Assignment: Ensure the key points are rotation invariant

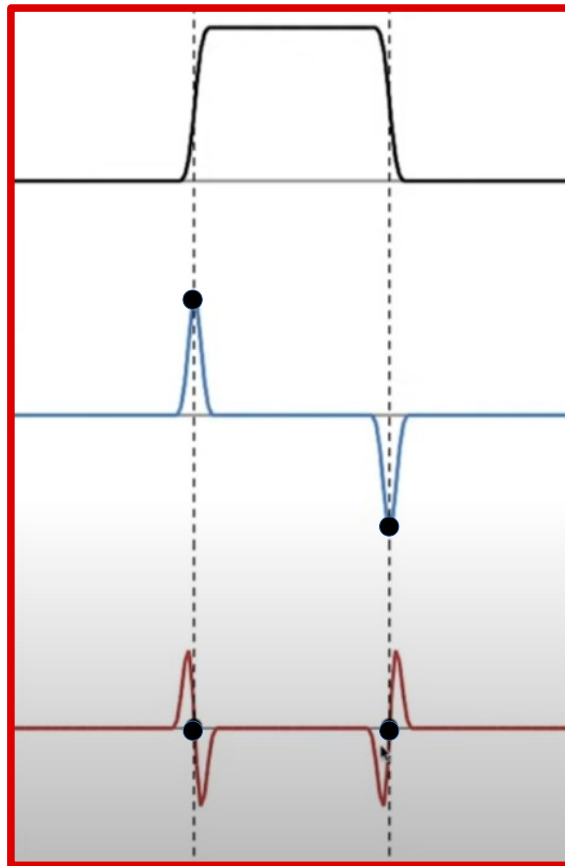
Review of Edge Detection using Gradients

First derivative: $\frac{\partial f}{\partial x}$

Local Extrema points indicate Edges

Second derivative: $\frac{\partial^2 f}{\partial x^2}$

Zero Crossing points indicate Edges



Review: Smoothing of images using Average filter and Gaussian filter

12



Original



After Applying Average Filter

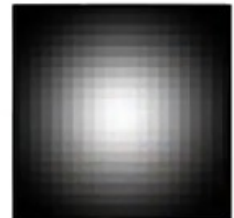
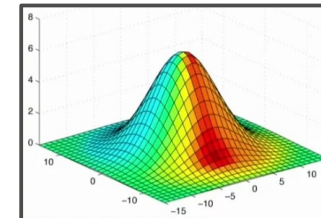


After Applying Gaussian Filter

Average Kernel

1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25

5x5

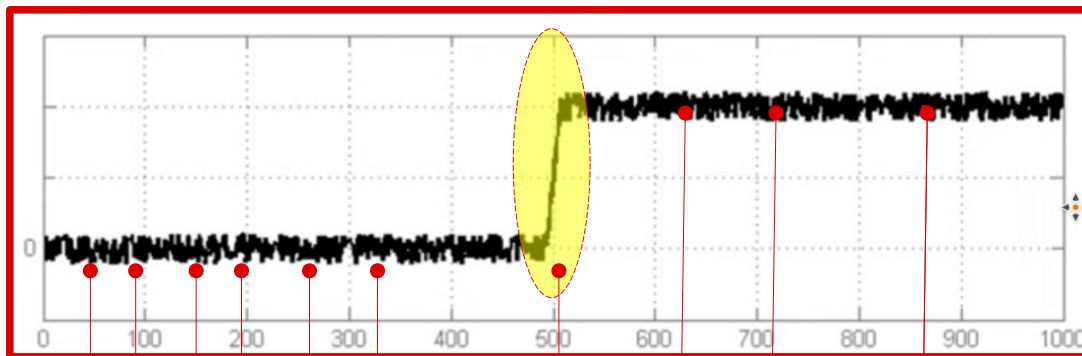


Gaussian Kernel

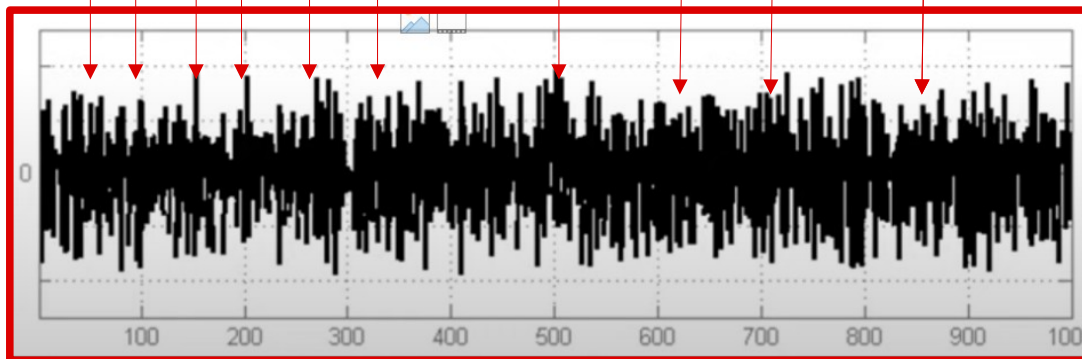
Review of Edge Detection using Gradients

Effect of Noise on detecting edges

Noisy Signal: $f(x)$



First derivative: $\frac{\partial f}{\partial x}$



Remember:

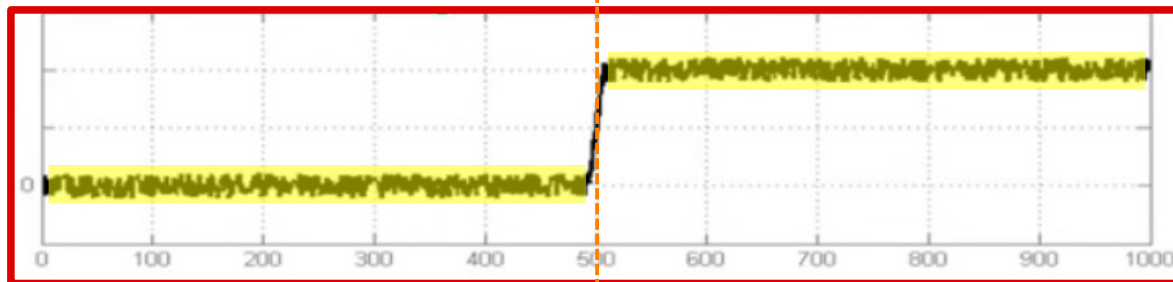
Gradient = $\tan \theta$

IT Depends on Angle not Length

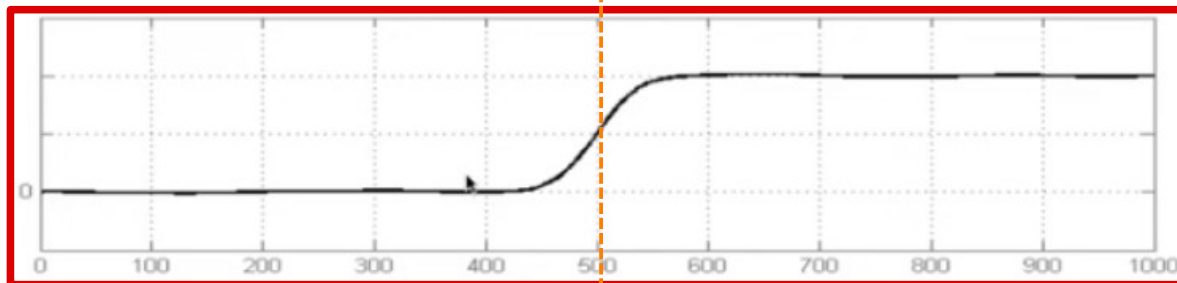
Conclusion: Apply Smoothing filters before calculating the gradient

Review of Edge Detection using Gradients

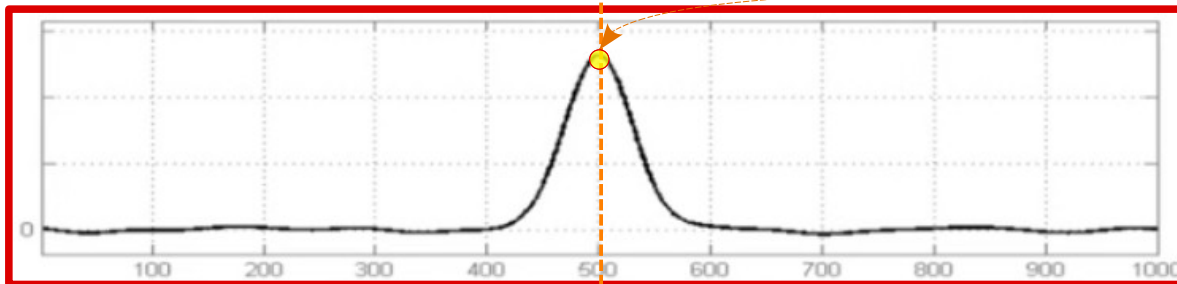
Noisy Signal: $f(x)$



Smoothed Signal: $f(x)$



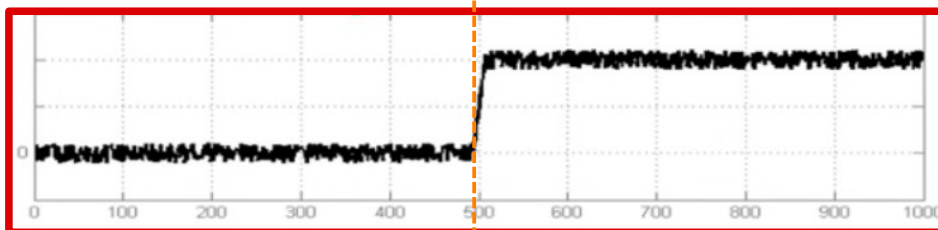
First derivative: $\frac{\partial f}{\partial x}$



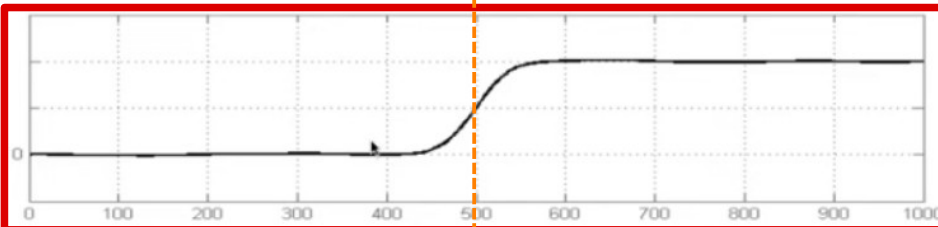
Detect
Peak

Review of Edge Detection using Gradients

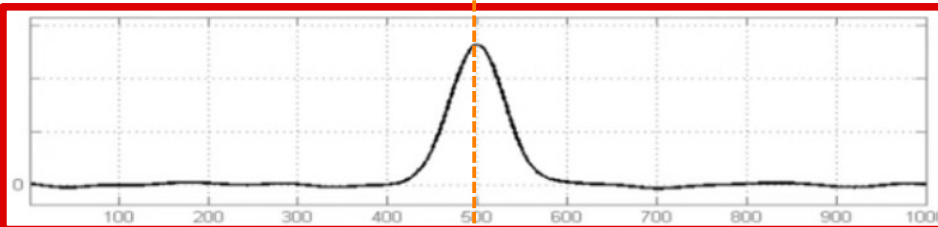
Noisy Signal: $f(x)$



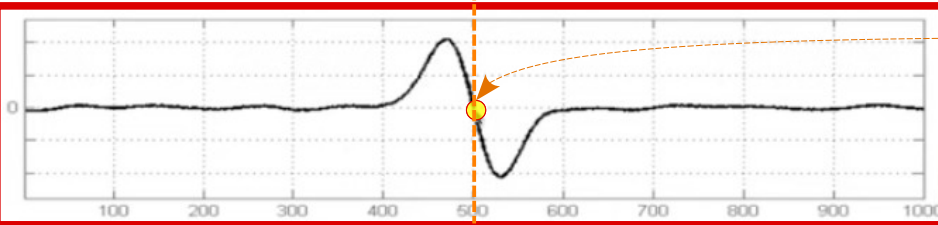
Smoothed Signal: $f(x)$



First derivative: $\frac{\partial f}{\partial x}$



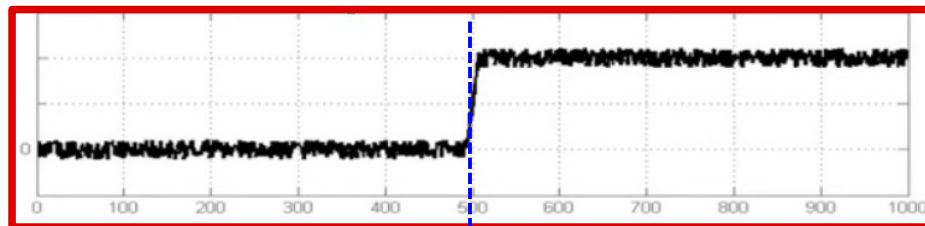
Second derivative: $\frac{\partial^2 f}{\partial x^2}$



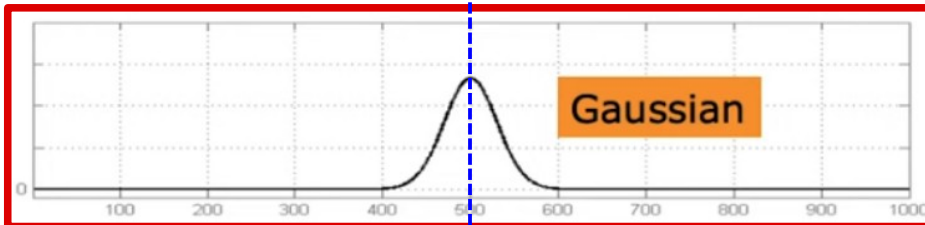
Detect
Zero Crossing

Review of Edge Detection using Gradients

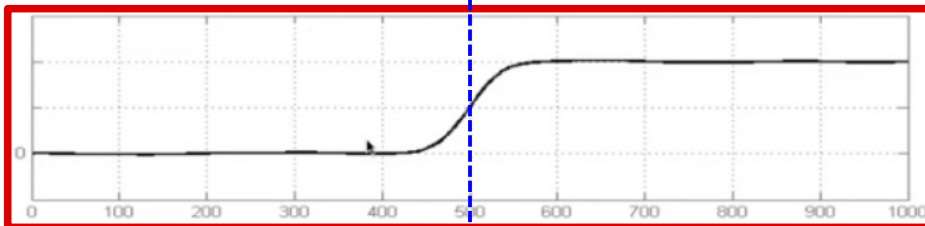
Noisy Signal: $f(x)$



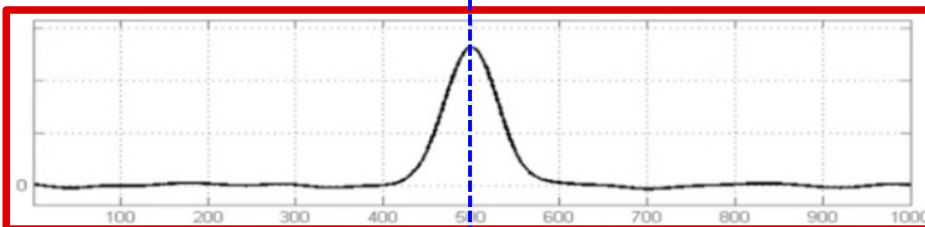
Gaussian Filter N_σ
(Smoothing Filter)



Convolution $N_\sigma * f(x)$
(Apply Smoothing Filter)

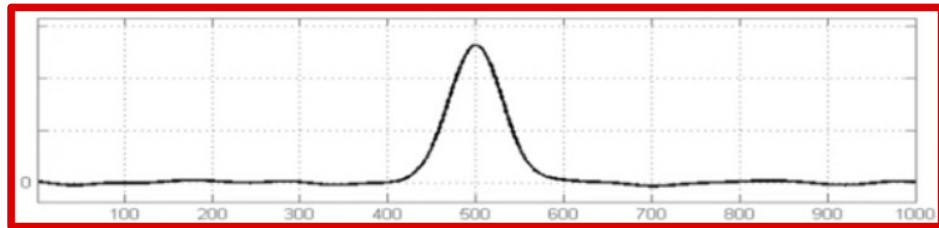


First derivative : $\frac{\partial (N_\sigma * f(x))}{\partial x}$
 $= \nabla (N_\sigma * f(x))$



Simplify Convolution and Derivative Operations

First derivative : $\frac{\partial (N_\sigma * f(x))}{\partial x}$
 $= \nabla (N_\sigma * f(x))$



Accomplished in two steps:

Filter $f(x)$ with N_σ (convolution)
 then calculate **derivative** of Output

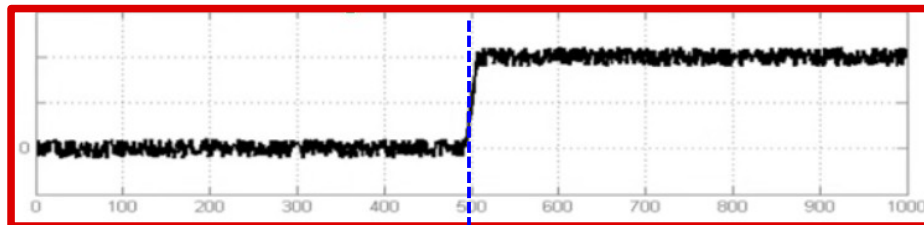
Since first derivative is a linear operation.
 And Gaussian smoothing is linear operation

$$\nabla (N_\sigma * f(x)) = \nabla (N_\sigma) * f(x)$$

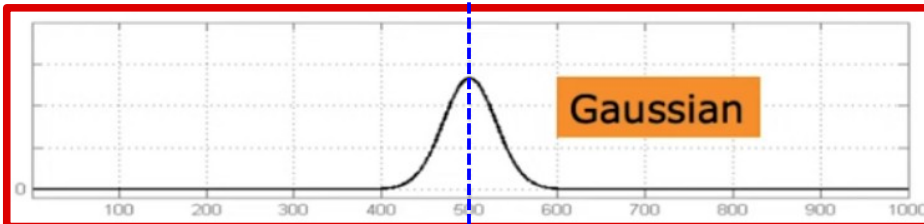
Convolute using **first Derivative of Gaussian Filter**
 (One Operation)

Review of Edge Detection using First Derivative

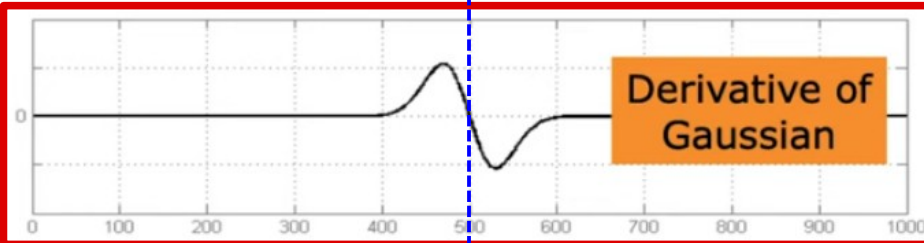
Noisy Signal: $f(x)$



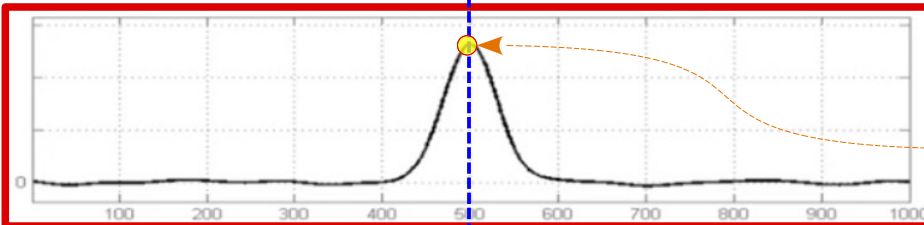
Gaussian Filter N_σ
(Smoothing Filter)



Derivative of N_σ $\nabla(N_\sigma)$



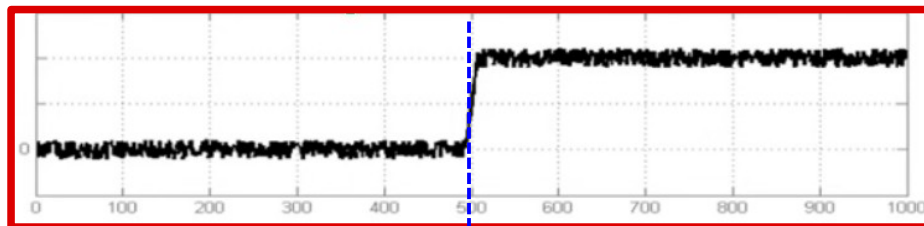
First derivative: $\nabla(N_\sigma) * f(x)$



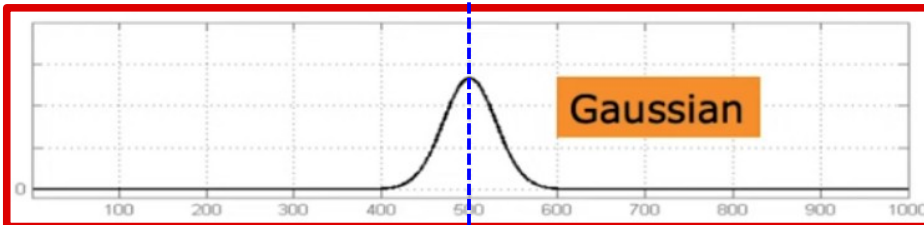
Detect Peak

Review of Edge Detection using Second Derivative

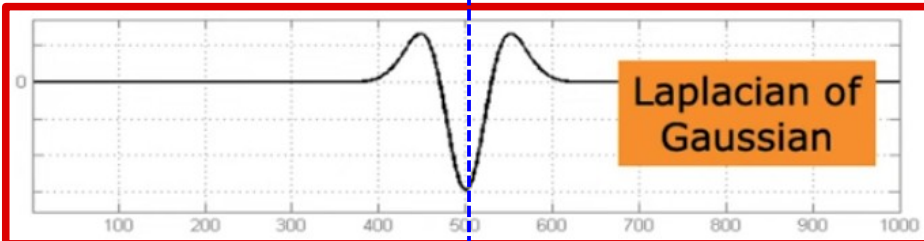
Noisy Signal: $f(x)$



Gaussian Filter N_σ
(Smoothing Filter)

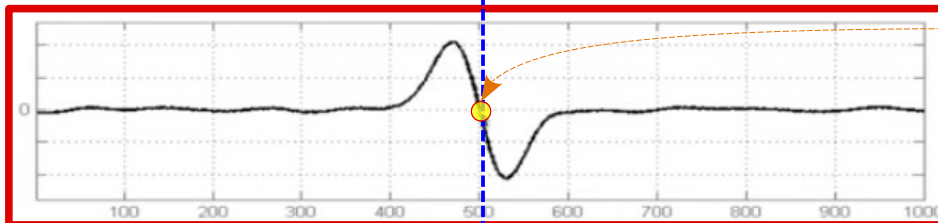


Laplacian of N_σ $\nabla^2(N_\sigma)$



Second derivative: $\nabla^2(N_\sigma) * f(x)$

Equivalent to $\nabla^2(N_\sigma * f(x))$



Detect
Zero Crossing

Laplacian of Gaussian (LOG)

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

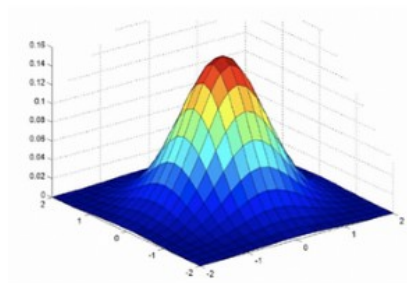


$$\text{LoG} = -\frac{1}{\pi\sigma^4} \left[1 - \frac{x^2+y^2}{2\sigma^2} \right] e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

**Gaussian Kernel
(2-D)**

0	-1	0
-1	4	-1
0	-1	0

**Traditional
Laplacian Filter
(Edge Detection)**



**Gaussian Filter
(Low Pass Filter)**



0	1	1	2	2	2	1	1	0
1	2	4	5	5	5	4	2	1
1	4	5	3	0	3	5	4	1
2	5	3	-12	-24	-12	3	5	2
2	5	0	-24	-40	-24	0	5	2
2	5	3	-12	-24	-12	3	5	2
1	4	5	3	0	3	5	4	1
1	2	4	5	5	5	4	2	1
0	1	1	2	2	2	1	1	0

**Laplacian of Gaussian Filter
(Smooth the Image then Detect Edges)**

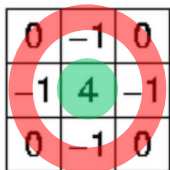
Laplacian of Gaussian (LOG)

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

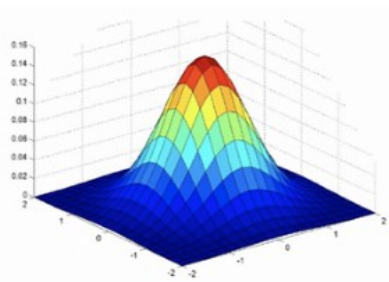


$$\text{LoG} = -\frac{1}{\pi\sigma^4} \left[1 - \frac{x^2+y^2}{2\sigma^2} \right] e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

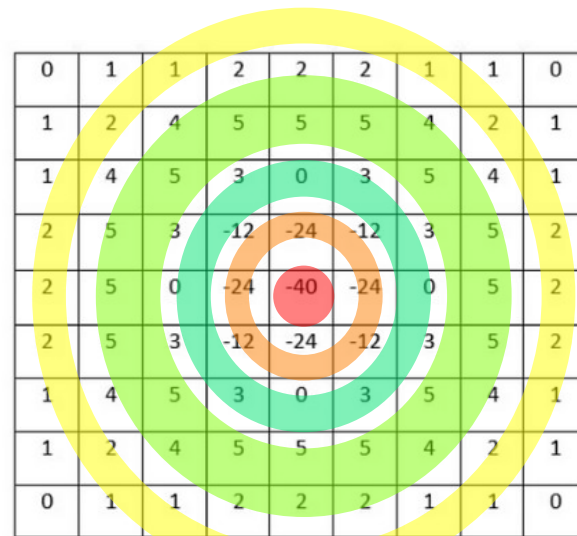
**Gaussian Kernel
(2-D)**



**Traditional
Laplacian Filter
(Edge Detection)**

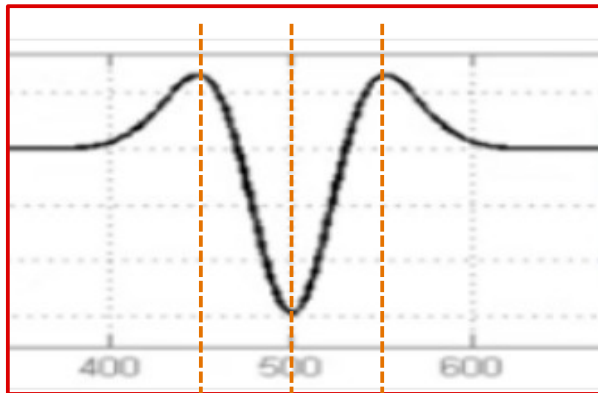


**Gaussian Filter
(Low Pass Filter)**



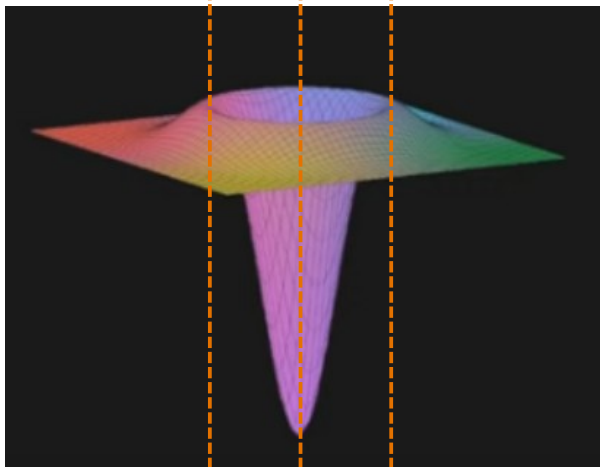
**Laplacian of Gaussian Filter
(Smooth the Image then Detect Edges)**

Laplacian of Gaussian (LOG)



Laplacian of Gaussian $\nabla^2(N_\sigma)$

1-D: $\frac{\partial^2(N_\sigma)}{\partial x^2}$



Laplacian of Gaussian $\nabla^2(N_\sigma)$

2-D: $\frac{\partial^2(N_\sigma)}{\partial x^2} + \frac{\partial^2(N_\sigma)}{\partial y^2}$

(Mexican Hat)

$$\text{LoG} = -\frac{1}{\pi \sigma^4} \left[1 - \frac{x^2 + y^2}{2\sigma^2} \right] e^{-\left(\frac{x^2 + y^2}{2\sigma^2} \right)}$$

LOG and Normalized LOG (NLOG)

Cases:

1

2

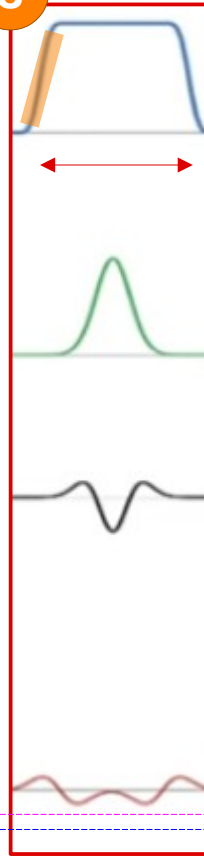
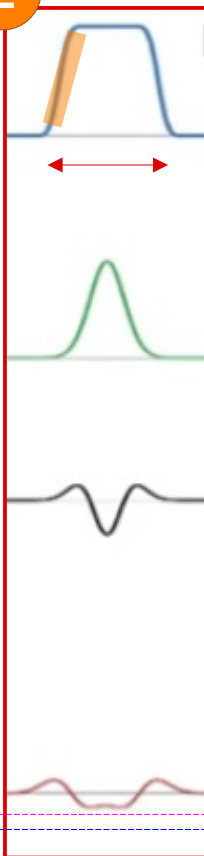
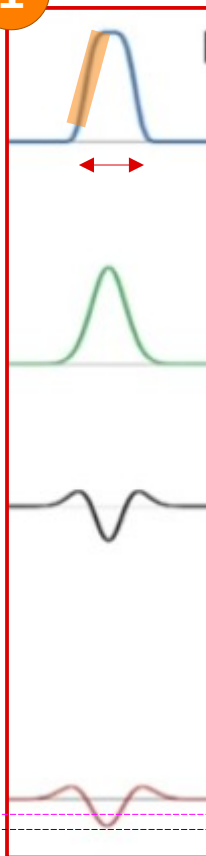
3

$f(x)$

N_σ

$\frac{\partial^2 N_\sigma}{\partial x^2}$

$\frac{\partial^2 N_\sigma}{\partial x^2} * f(x)$



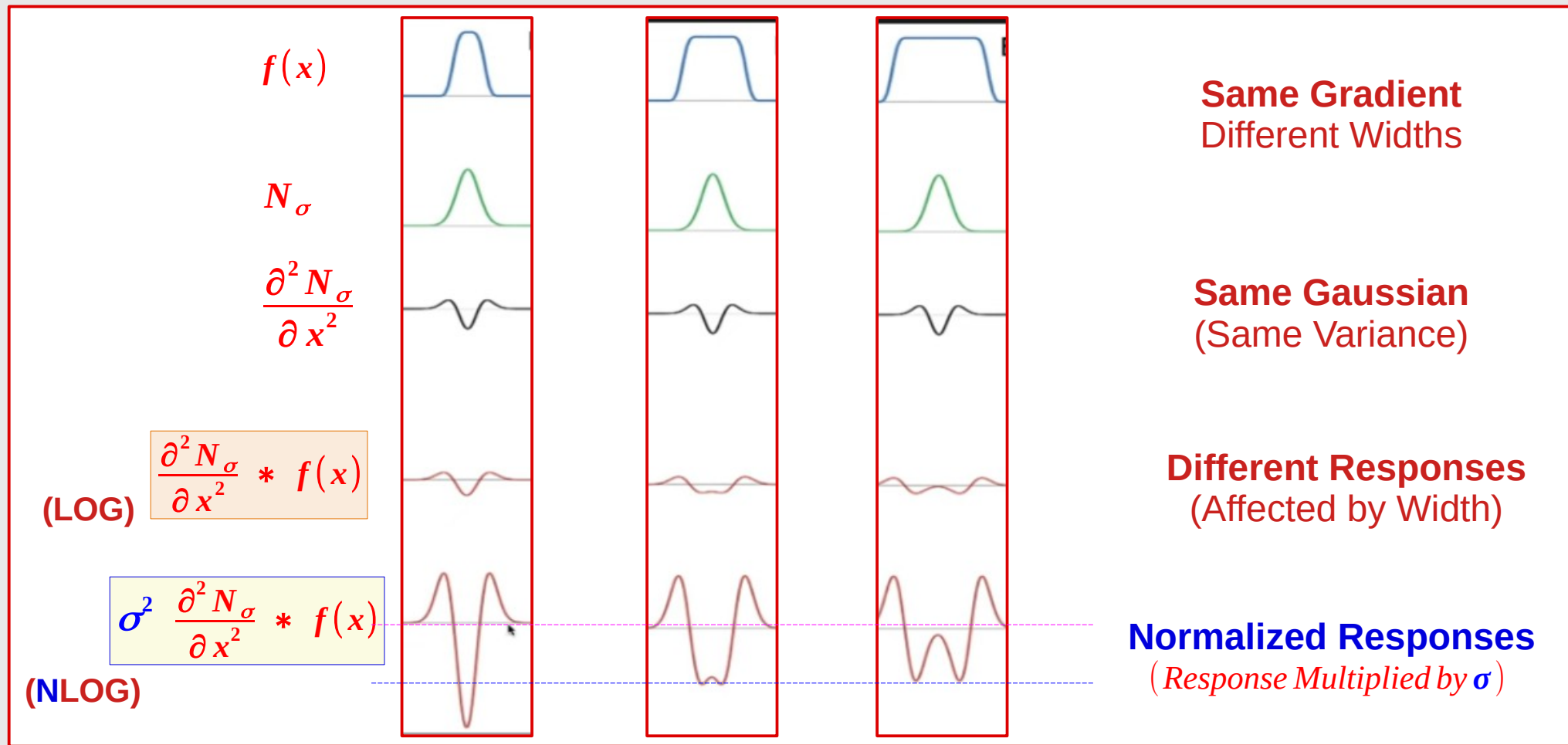
Same Gradient
Different Widths

Same Gaussian
(Same Variance)

Same Gaussian
Derivative

Responses decrease
(*Affected by Width*)

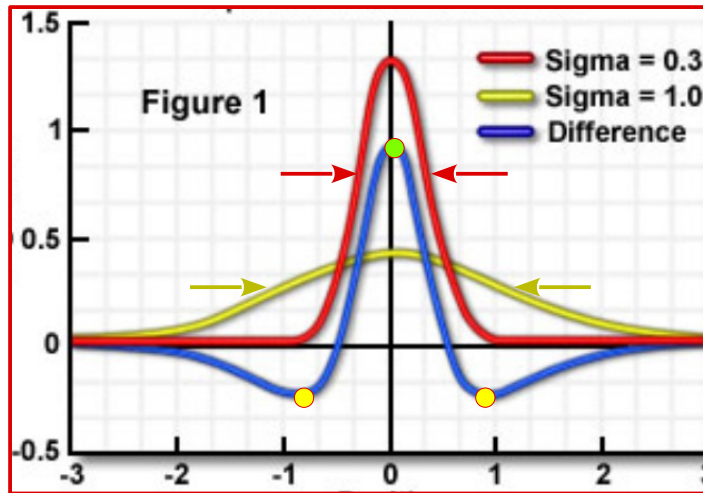
LOG and Normalized LOG (NLOG)



Fast Approximation of (NLoG) with Difference of Gaussian (DoG)

25

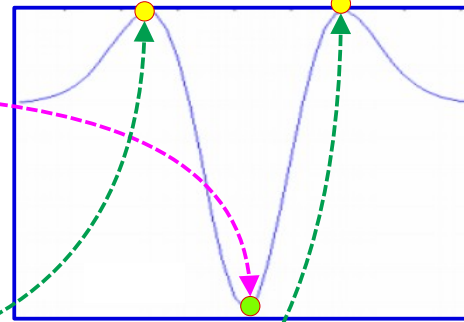
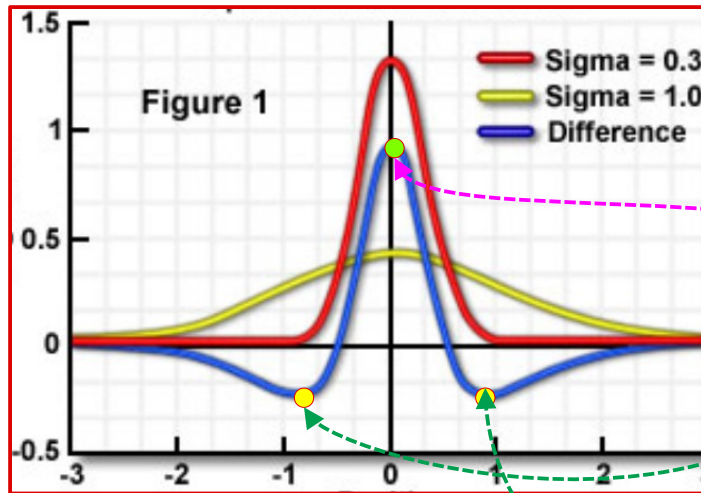
Normal Distributions
with different scales σ
of $\sigma = 0.3$ and $\sigma = 1.0$



$$\text{Diff. of Gaussian} = N_{\sigma=0.3} - N_{\sigma=1.0}$$

Fast Approximation of (NLoG) with Difference of Gaussian (DoG)

Normal Distributions
with different scales σ
of $\sigma = 0.3$ and $\sigma = 1.0$



$$\text{Diff. of Gaussian} = N_{\sigma=0.3} - N_{\sigma=1.0}$$

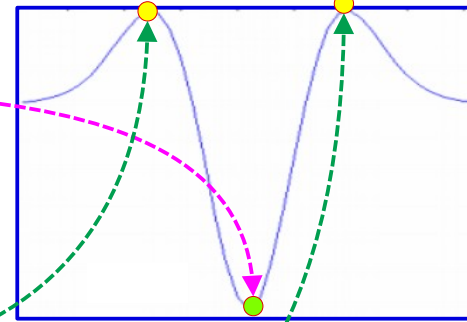
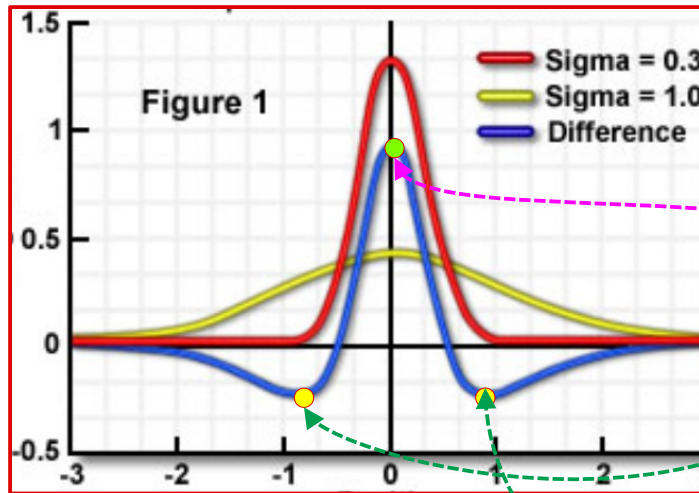
$$\text{Diff. of Gaussian} = N_{\sigma=1.0} - N_{\sigma=0.3}$$

$$\text{DoG} (N_{\text{higher } \sigma} - N_{\text{lower } \sigma})$$

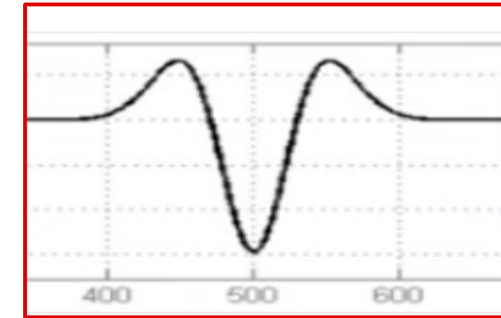
Fast Approximation of (NLoG) with Difference of Gaussian (DoG)

27

Normal Distributions
with different scales σ
of $\sigma = 0.3$ and $\sigma = 1.0$



$\approx$



Normalized Laplacian of Gaussian
(Can be approximated by DoG)

$$\text{Diff. of Guassian} = N_{\sigma=0.3} - N_{\sigma=1.0}$$

$$\text{Diff. of Guassian} = N_{\sigma=1.0} - N_{\sigma=0.3}$$

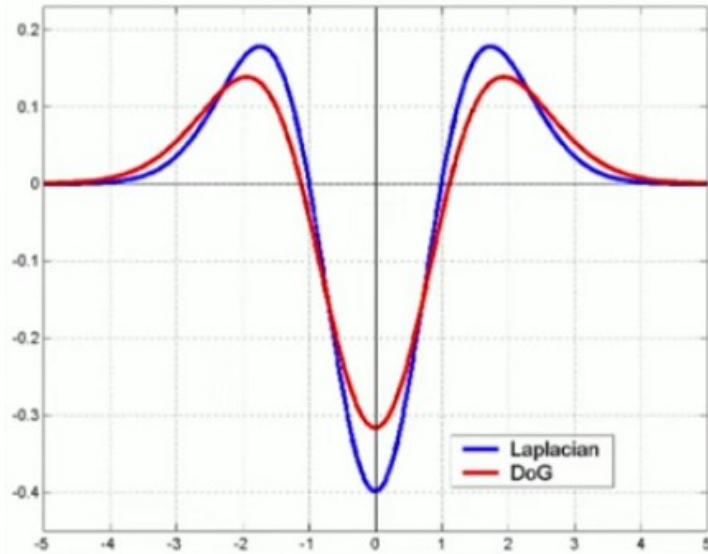
$$\text{DoG} (N_{\text{higher } \sigma} - N_{\text{lower } \sigma}) \approx$$

Good Choice if (higher $\sigma = 2 * \text{lower } \sigma$)

NLoG

Fast Approximation of (NLoG) with Difference of Gaussian (DoG)

28



Laplacian of Gaussian

$$\nabla^2(N_\sigma)$$

Normalized Laplacian of Gaussian

$$\sigma^2 \nabla^2(N_\sigma)$$

$$N_{2\sigma} - N_\sigma \approx NLoG$$

In general:

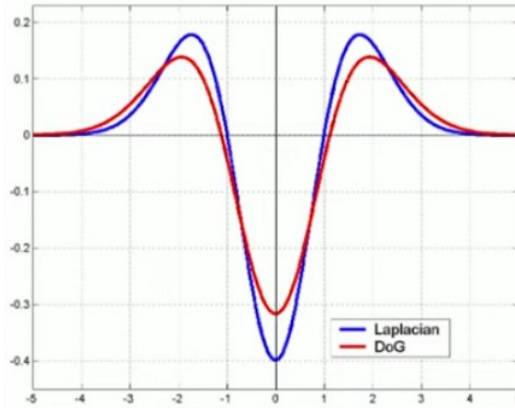
$$\text{Difference of Gaussian (DoG)} = N_{S\sigma} - N_\sigma \approx (S-1)NLoG \approx (S-1)\sigma^2 \nabla^2 N_\sigma$$

“S” is a scaling factor (e.g. 2,3,4,5,...)

$$DoG \approx (S-1)NLog$$

Fast Approximation of (NLoG) with Difference of Gaussian (DoG)

29



Laplacian of Gaussian

$$\nabla^2(N_\sigma)$$

Normalized Laplacian of Gaussian

$$\sigma^2 \nabla^2(N_\sigma)$$

$$\text{Difference of Gaussian (DoG)} = N_{s\sigma} - N_\sigma \approx (s-1) \sigma^2 \nabla^2 N_\sigma \approx (s-1) \text{NLoG}$$

$$\text{DoG} \approx (s-1) \text{NLoG}$$

$$\text{for } s=2 \Rightarrow N_{2\sigma} - N_\sigma \approx \text{NLoG}$$

Conclusion:

Apply Normalized Laplacian of Gaussian (**NLoG**) (with variance σ) on an image
is **almost equivalent to**

Subtracting two Filtered images with Gaussian filters of variances σ and 2σ

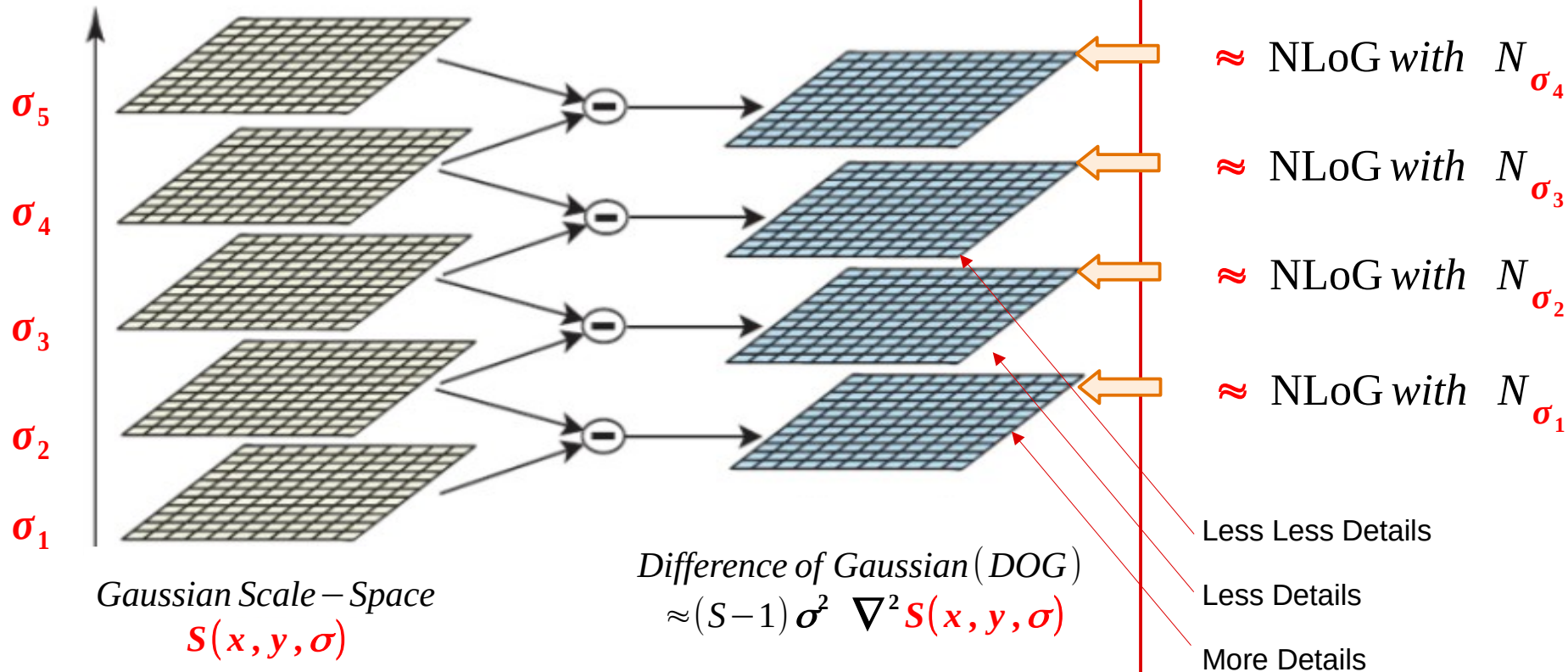
Fast Approximation of (NLoG) with Difference of Gaussian (DoG)

Conclusion:

- **NLoG** can be approximated to **DoG** of two Gaussian with scale difference
- Rather than computing the normalized Laplacian of Gaussian, one can simply take the stack of images, which is just the original image smoothed with different values of sigma
- Then take any two consecutive images and find the difference between them,

you actually get the result you would have gotten if you had applied the normalized Laplacian of Gaussian to the first of these images

Fast Approximation of (NLoG) with Difference of Gaussian (DoG)



Effect of Different variance values


 σ_1

 σ_2

 σ_3

 σ_4

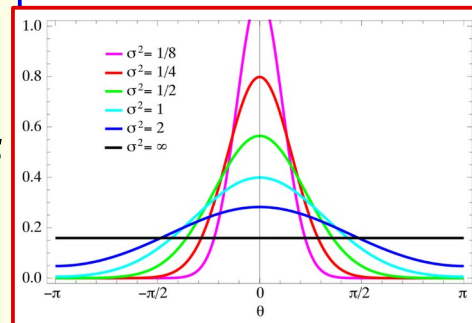
Smaller variance may result in partial removal of noise

⇒ Some noise may be detected as edges

Larger variance may result in removal of important edges and corners

⇒ losing some blobs

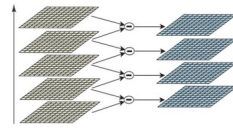
What is the best σ value to filter the image with ?



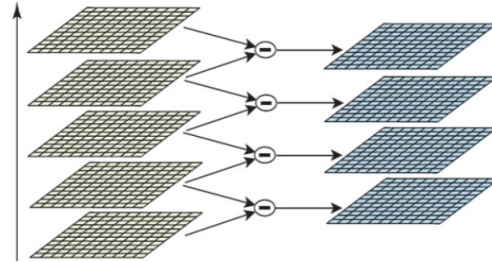
Difference of Gaussian (DoG) for Multiple Image Sizes (Octaves)



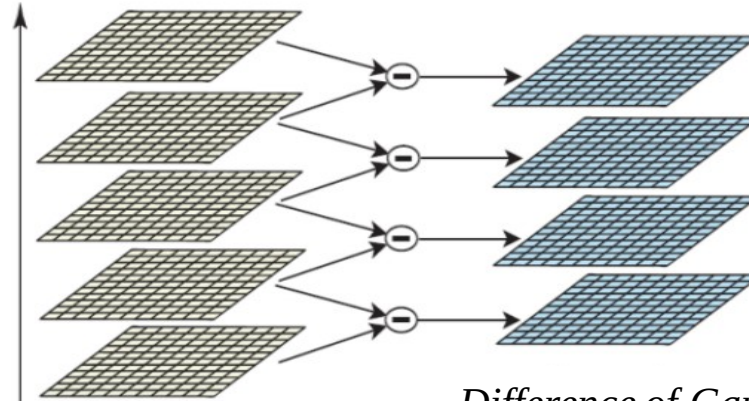
Third Octave
(multiple Scales)



Second Octave
(multiple Scales)



First Octave
(multiple Scales)



Gaussian Scale – Space
 $S(x, y, \sigma)$

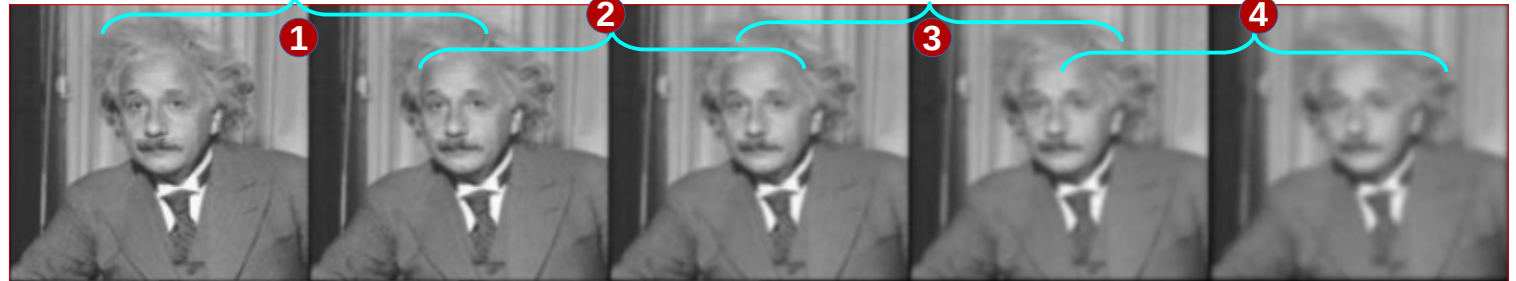
Difference of Gaussian (DOG)
 $\approx (S-1) \sigma^2 \nabla^2 S(x, y, \sigma)$

Difference of Gaussian (DoG)

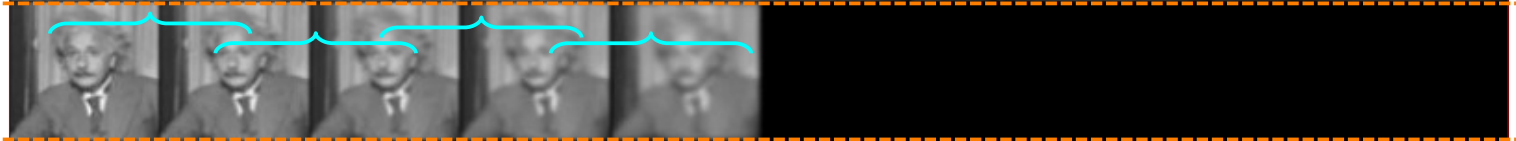
34

Gaussian Blurred Images
Multiple Scales

First Octave



Second Octave



Third Octave



Fourth Octave



Diff. Of Gaussian (DoG)

First Octave



Second Octave



Third Octave



Fourth Octave



Effect of Different variance values

 σ_1

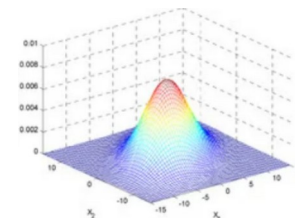
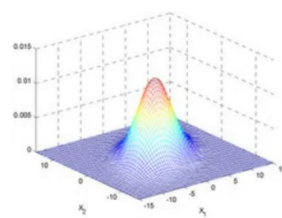
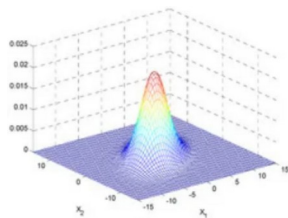
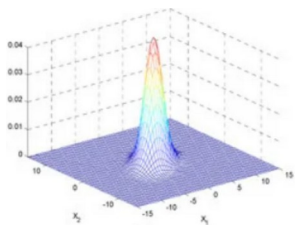
<

 σ_2

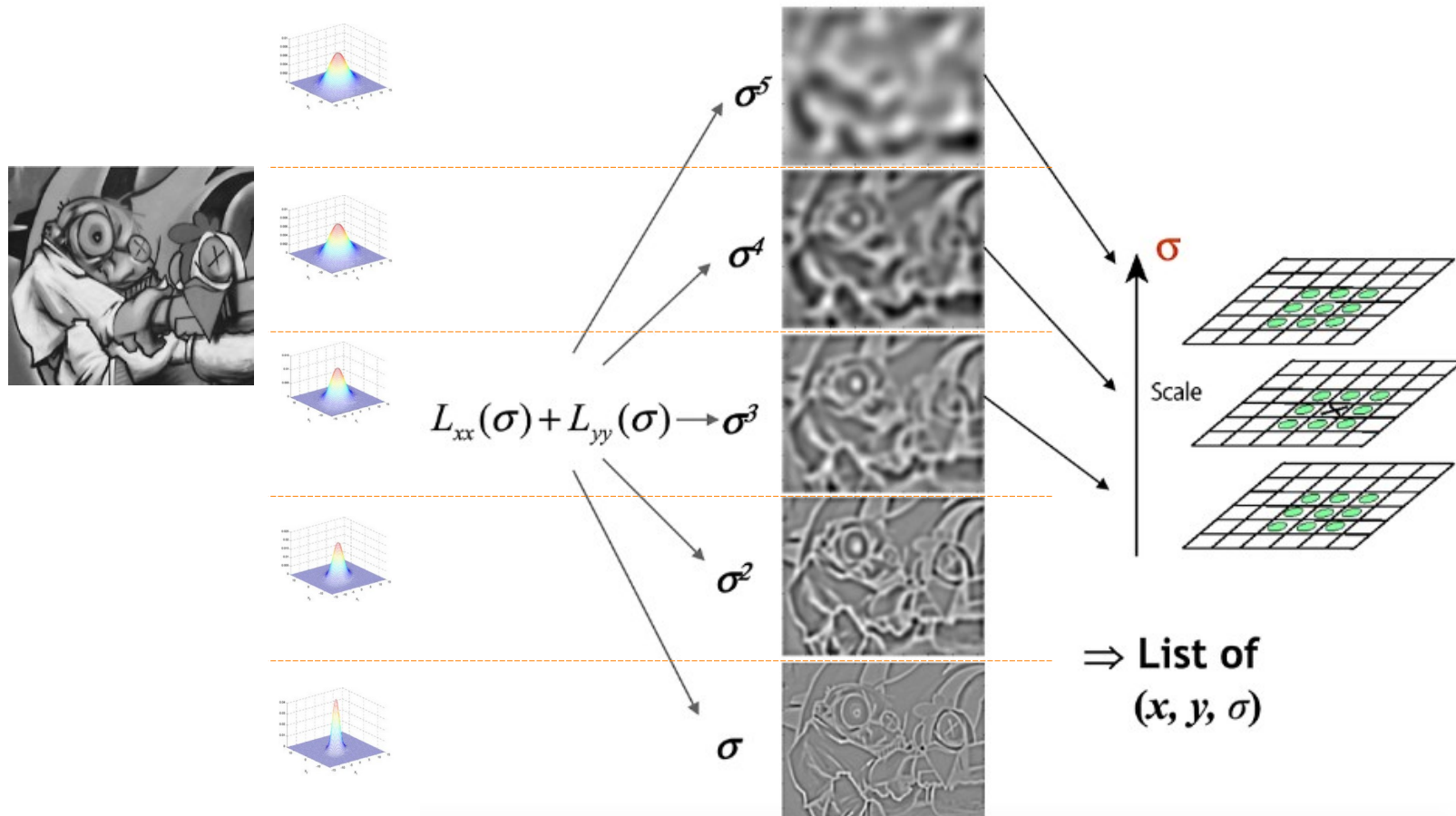
<

 σ_3

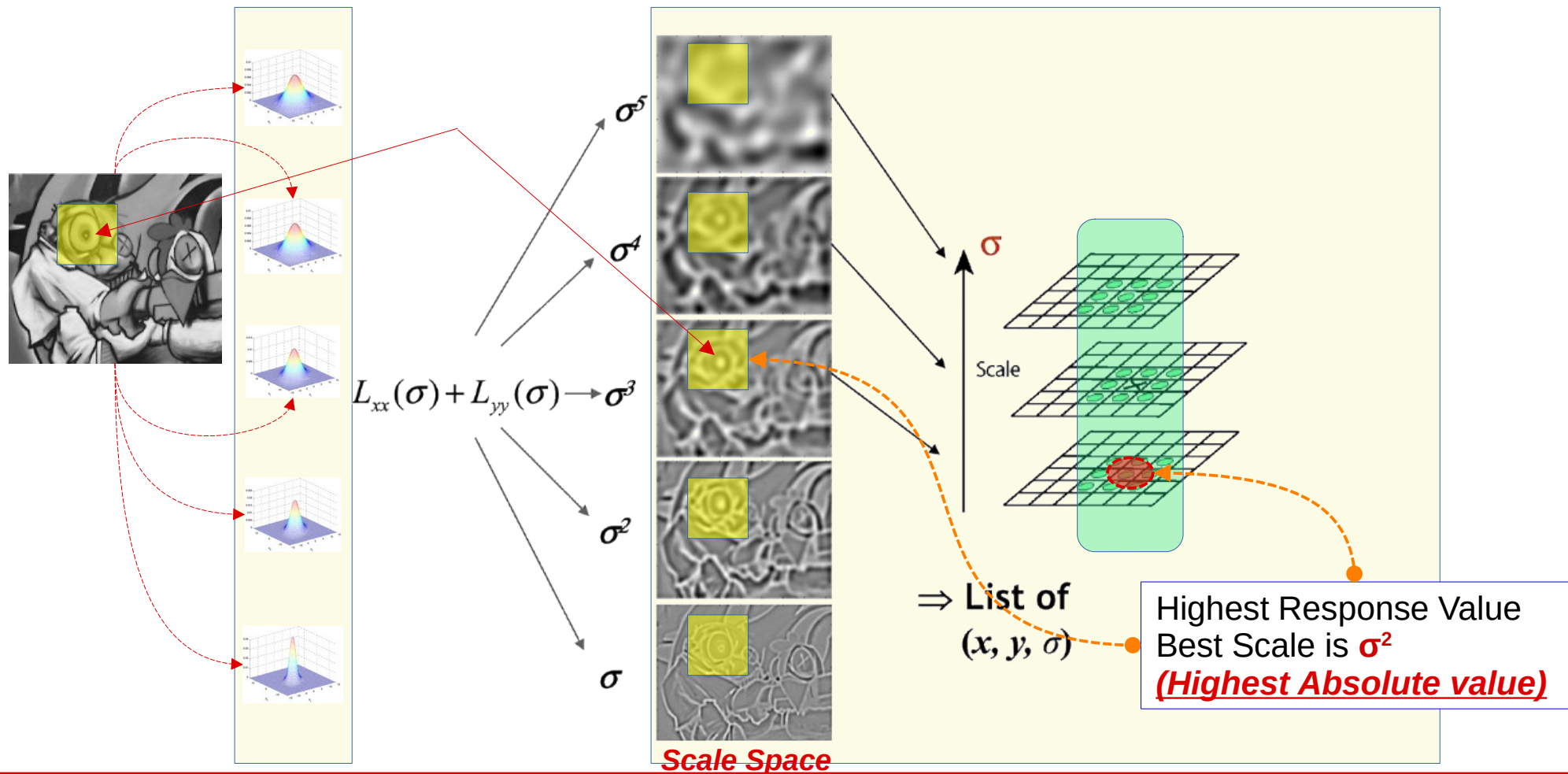
<

 σ_4 

Selecting Best Scale of σ value



Selecting Best Scale of σ value



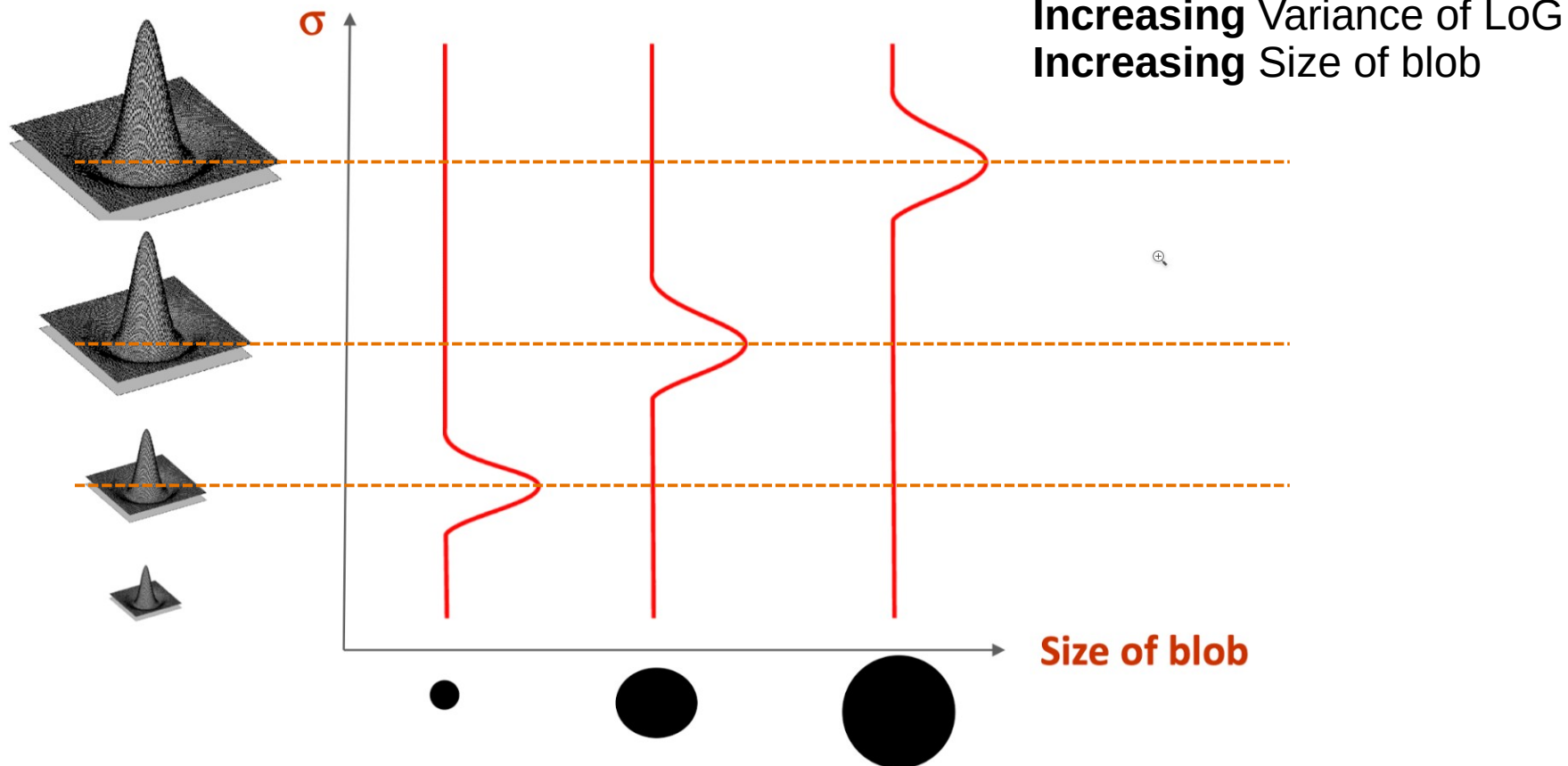
Scale-Space Blob Detector

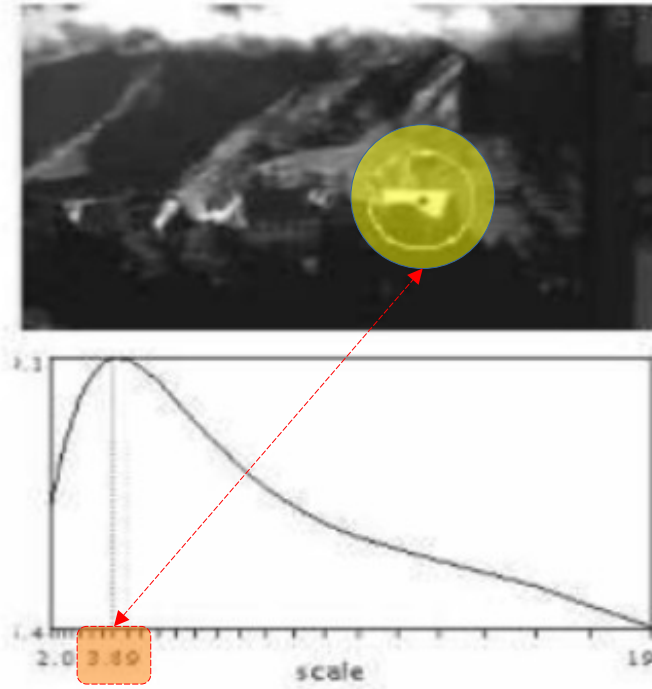
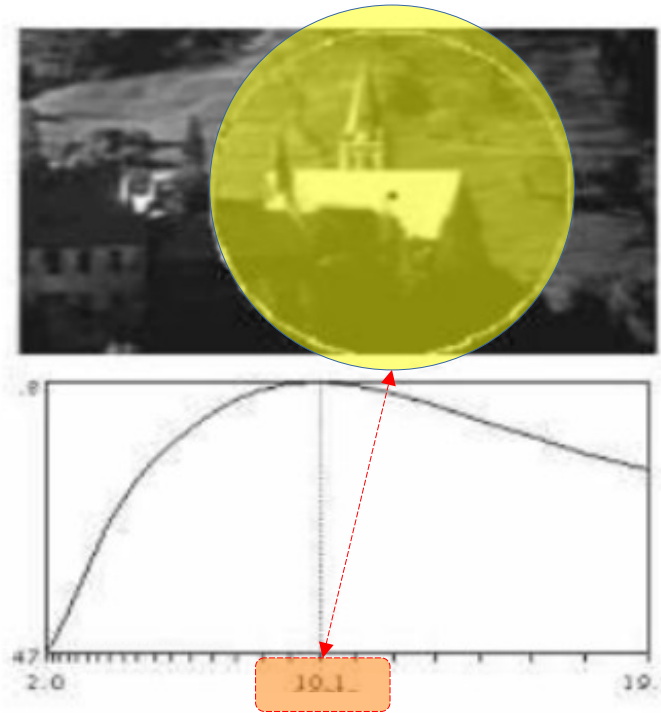
Apply **NLoG** filters of multiple scales (of Sigma) to create a scale space. (Using **DoG**) to speed up processing

Find the local maximum in the scale space,

This maximum is a **blob** that represents important features at a specific scale of the image.

Automatic Scale selection





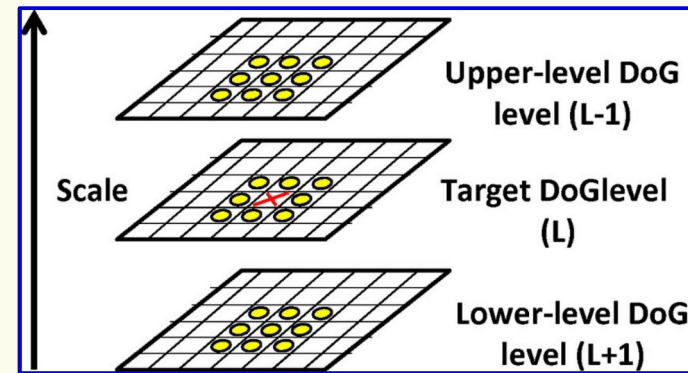
Increasing Variance of LoG
Increasing Size of blob

[1] Extracting SIFT Interest Points

[1] Detect Initial Extrema (Max or Min)

For each DoG image, identify potential key points by detecting local **maxima** and **minima**.

- Each pixel in the DoG image is compared to its **26 neighbors** in the 3D scale-space:
- *Remember: **Scale-Space** is obtained using (same octave size – different variances)*
 - 8 neighbors in the current image.
 - 9 neighbors in the image at the scale above.
 - 9 neighbors in the image at the scale below.



- If the pixel value is either greater or smaller than all of its neighbors, it is considered a local extremum and a candidate key point.

Extracting SIFT Interest Points

[2] Refinement of Key Point Location

Objective of refinement is to achieve:

sub-pixel and

sub-scale accuracy,

Refinement can be achieved by refine the initial key point location using a **Taylor series expansion** of the DoG function around the candidate point:

$$D(x) = D + \frac{\partial D}{\partial x} x + \frac{1}{2} x^T \frac{\partial^2 D}{\partial x^2} x$$



$$X = - \left(\frac{\partial^2 D}{\partial x^2} \right)^{-1} + \frac{\partial D}{\partial x}$$

Where:

D is the DoG value at the initial key point

$\frac{\partial D}{\partial x}$ is the gradient of the DoG

$\frac{\partial^2 D}{\partial x^2}$ is the Hessian matrix (second – order derivatives)

x is the offset from the initial key point location

Solving for x gives the exact location of the extremum

Update the key point location and scale using this refined value

This Part is not of our interest in this course. Please refer to Taylor series expansion for more details

Extracting SIFT Interest Points

[3] Filtering Unstable Keypoint

(3-1) Discarding Low Contrasted Extrema (Low Contrast Removal)

Image noise will typically produce several spurious DoG extrema.

Such extrema are:

unstable and

not linked to any particular structure in the image.

SIFT attempts to eliminate these false detections by discarding candidate keypoints with a **DoG value** below a threshold **C**

if $|\text{value}| < C$ then discard the candidate keypoint.

This filtering will obtain a more noise immune keypoints

(3-2) Discarding candidate keypoints on edges

Candidate keypoints lying on edges are difficult to precisely locate.

The 2-D Hessian matrix of DoG of the DoG provides a characterization of those keypoint candidates.

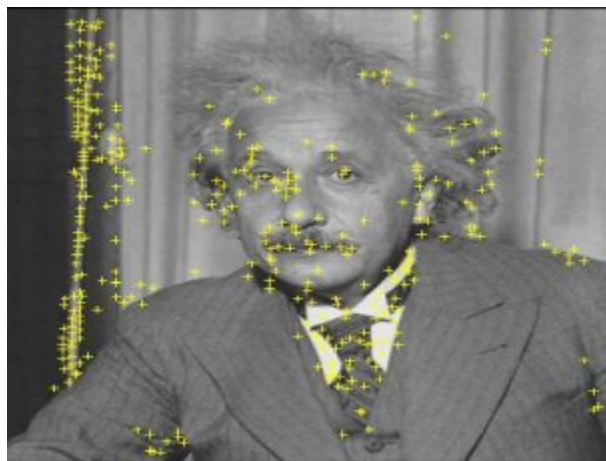
Use the Hessian matrix to eliminate key points that have strong responses along edges.

This Part is not of our interest in this course. Please refer to Taylor series expansion for more details

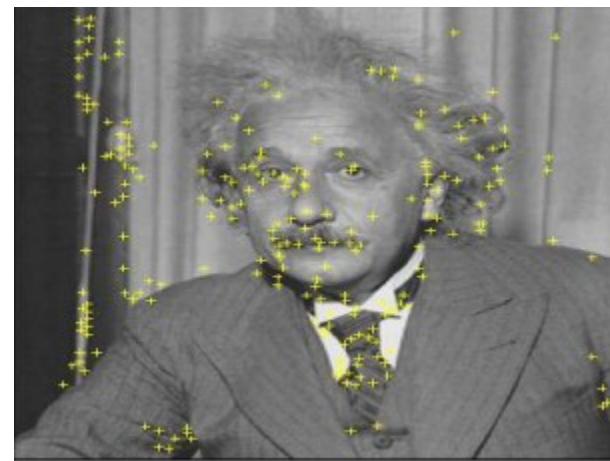
Extracting SIFT Interest Points



Initial Keypoints
(before refinements)



After removing
Low Contrast Points



After removing
Points on Edges

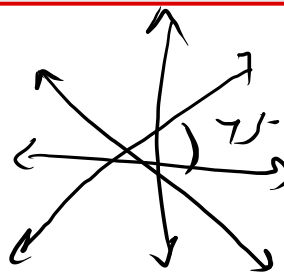
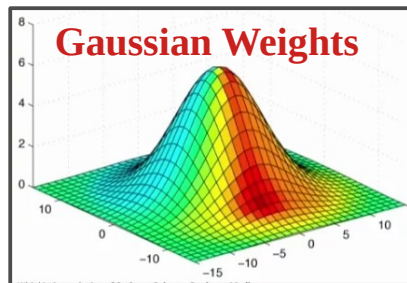
Compute Principle Orientation of each Keypoint

Steps:

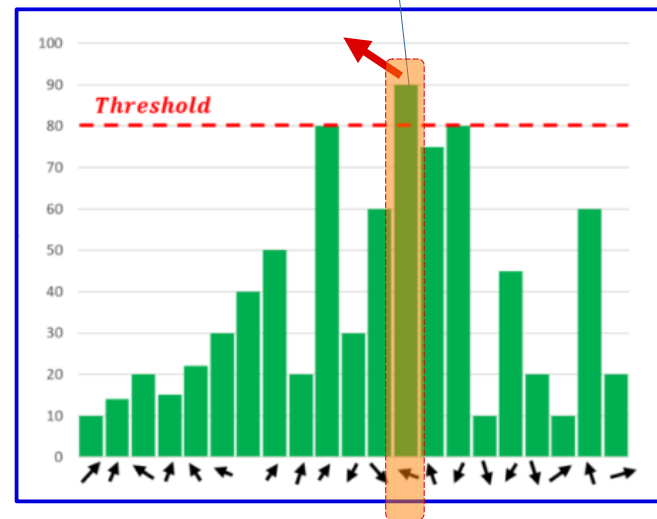
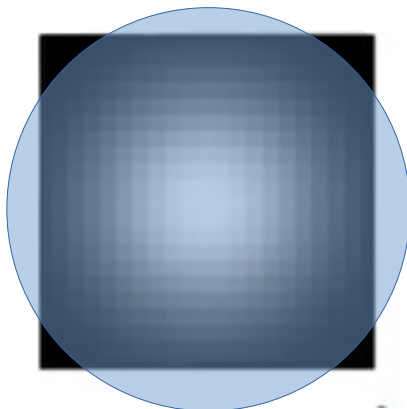
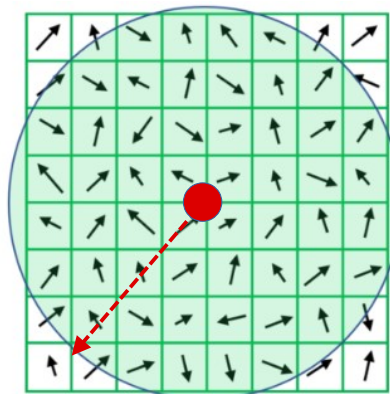
- Create a histogram of gradients for pixels around the keypoint's neighborhood.
- Define a square neighborhood around each Keypoint.
- Select Neighborhood radius = $3 * \text{Scale of the Keypoint}$. (Remember: scale is the sigma of the Gaussian distribution associated with the Keypointpoint)
- Compute the magnitude and orientation of the 2D gradient at each pixel in this neighborhood.
- Create a 36-bin histogram for the orientations (i.e 10 degrees per histogram bin).
- Though all pixels are neighbors to the Keypoint, some of them are closer to the Keypoint, those pixels should get more weight in the histogram over the fat pixels. This can be achieved by Gaussian weighting of the gradient magnitude of each neighbor pixel. This makes pixels farther from the keypoint have less of an influence on the histogram.

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

$$\theta(x, y) = \tan^{-1} \left(\frac{L(x, y+1) - L(x, y-1)}{L(x+1, y) - L(x-1, y)} \right)$$



Orientation of the Keypoint



Gradient with **Magnitude and Direction** of the Neighborhood of Keypoint

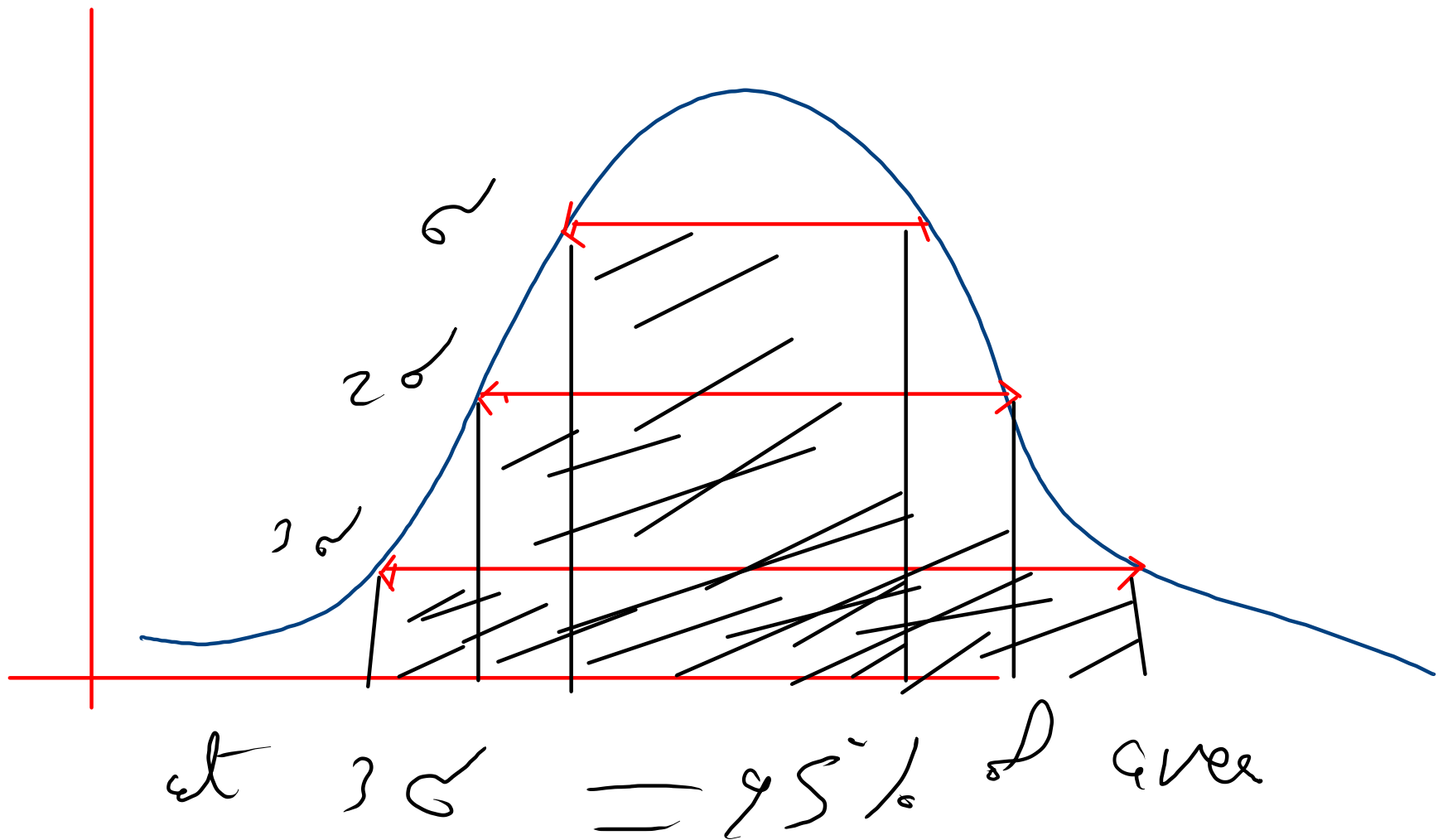
(Circle **Radius** is 3*scale of keypoint = **3σ**)

Weighting Gradients Magnitudes of neighbor pixels.

Assign higher weights to pixels closer to Keypoint

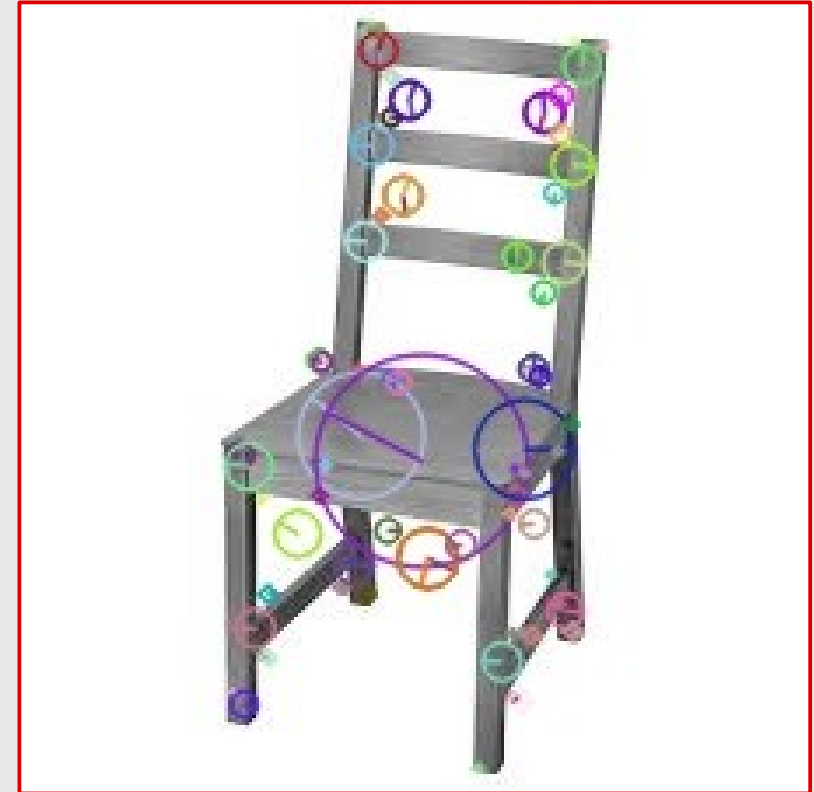
Histogram with 36 Pins (one pin for each 10 degrees)

Each pin is a **summation of weighted magnitudes** of gradients laying within the pin span.



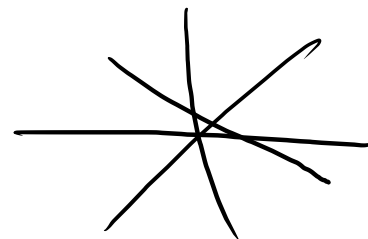
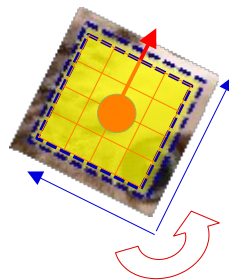
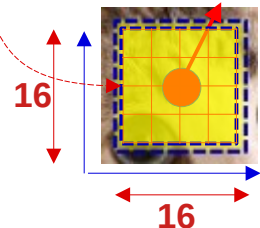
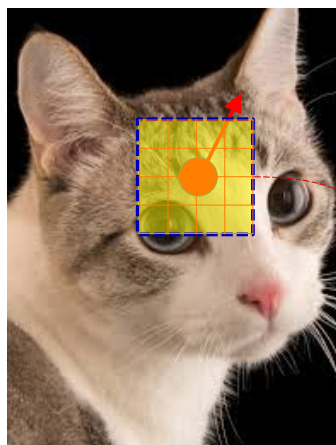
Visualization of Keypoint attributes (Location, Scale, and Orientation)

47



[2] Compute Keypoint Descriptors

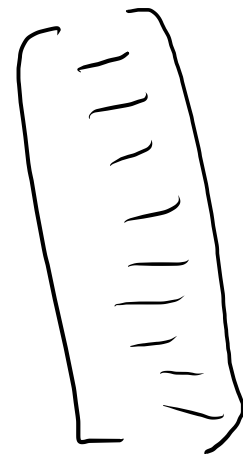
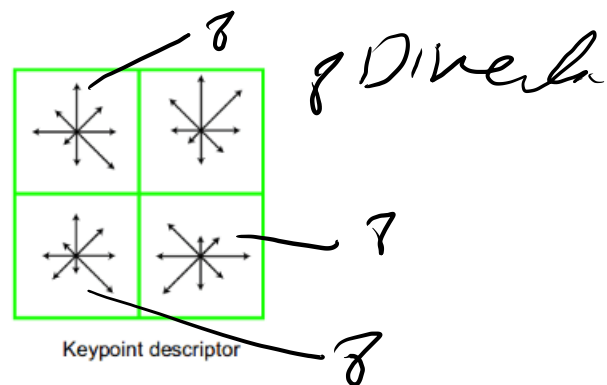
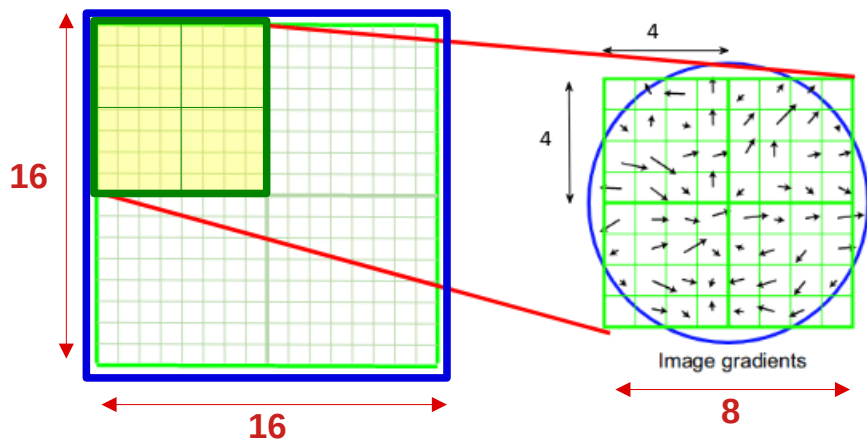
- Descriptors encode information about a keypoint's neighborhood and allow comparison between keypoints.
- For each keypoint, create another histogram of gradient orientations with only 8 bins to cover 360 degrees, instead of 36 bins as before.
- Consider a square neighborhood of size 16x16 pixels around each keypoint.
- Rotate this neighborhood by the keypoint's angle. This is what makes SIFT invariant to rotation.



Compute Keypoint Descriptors

- After rotating the 16x16 neighborhood by the keypoint's angle. Divide the neighborhood into 16 sub-blocks each of size 4x4.
- For each 4x4 sub-block (16 pixels) create another histogram of gradient orientations with only 8 bins to cover 360 degrees.
- Since there are 16 4x4 sub-blocks, a 16 histograms will be created each of 8 bins
- Concatenate the 16 histograms into one long vector of size $16 \times 8 = 128$ values

This is the descriptor vector corresponding to the Keypoint.



SIFT Properties – How to Achieve

50

[1] Scale Invariance:

Detects features at different scales of the image, making it robust to changes in size.

This is achieved by constructing a scale-space representation using Gaussian blurring and detecting features across multiple scales.

[2] Rotation Invariance:

Features are detected independently of the image orientation.

SIFT calculates the dominant orientation of each key point based on local gradients. Descriptor is aligned to this dominant orientation.

This allows the same feature to be identified even if the image is rotated.

SIFT Properties – How to Achieve

[3] Affine Transform Invariance:

Handles distortions such as scaling and rotation simultaneously.

This is due to handling both Scale invariance and Rotation invariance at the same time

[4] 3D Viewpoint Invariance:

The features remain robust even with changes in the 3D viewpoint.

The local features detected are largely unaffected by small changes in 3D viewpoint.

[5] Illumination Invariance:

Changes in lighting conditions do not significantly impact feature detection.

SIFT uses local image gradients, which are less sensitive to uniform changes in illumination

SIFT Properties – How to Achieve

52

[6] Occlusion Invariance:

Handles partial occlusions of objects within the scene.

Even if a part of the object is occluded or covered, SIFT can still detect and match the visible features, as each feature is localized and does not depend on global information.

SIFT Code

Using Open CV

Use opencv >=4.4

```
import cv2
import numpy as np
import os
```

Loading the image

```
path="/media/khaled/My_Home/Courses/Khaled/ZEWAIL/Spring/2025/DSAI 352 Computer Vision/Images/"
image_name="Chair_1"+".jpeg"
output_file="Chair_1"+"_outpout.jpeg"
image_with_path=os.path.join(path,image_name)
output_with_path=os.path.join(path,output_file)
img = cv2.imread(image_with_path)
```

```
gray= cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
```

Converting image to gray scale

```
sift = cv2.SIFT_create()
```

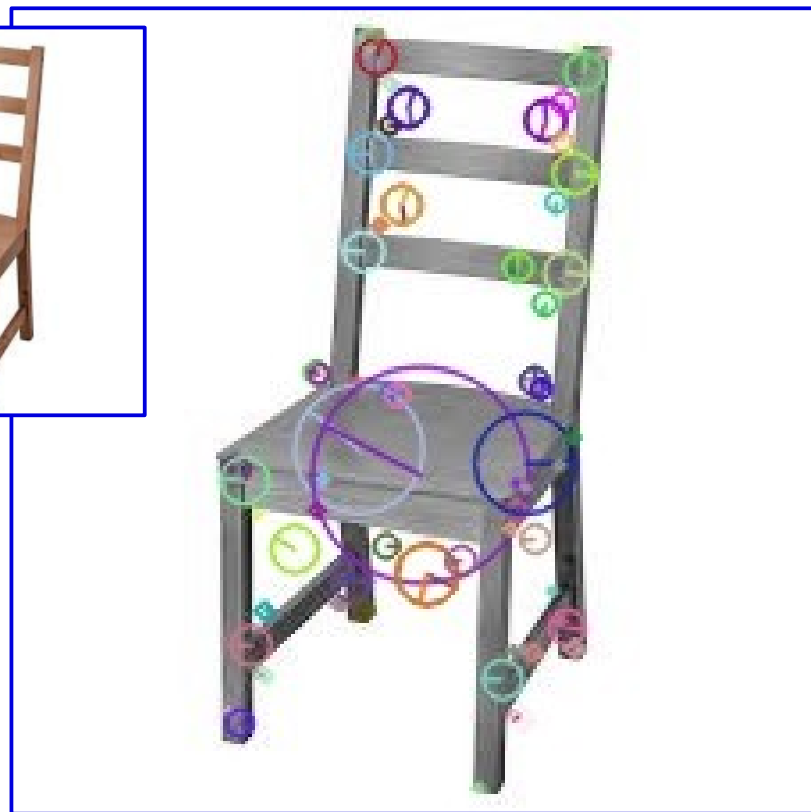
Create SIFT instance

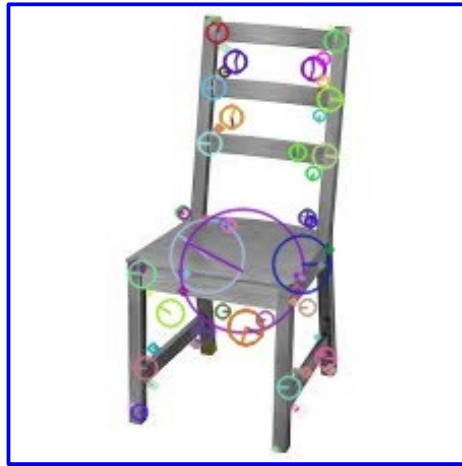
```
kp,des = sift.detectAndCompute (gray, None)
```

detect Keypoints and Descriptors

Marking the keypoint on the image using circles

```
img=cv2.drawKeypoints(gray , kp , img , lags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imwrite(output_with_path, img)
```





```
import cv2
import os
```

Loading the image

```
path="/media/khaled/My_Home/Courses/Khaled/ZEWAIL/Spring/2025/DSAI 352 Computer Vision/Images/"
img_format=".png"
image_name1="Quran_2"
image_name2="Quran_2_zoomed_clipped"
image_1_with_path=os.path.join(path,image_name1)+img_format
image_2_with_path=os.path.join(path,image_name2)+img_format
```

```
output_file="Matched_"+image_name1+"_"+image_name2+".jpg"
output_with_path=os.path.join(path,output_file)
```

Reading images, convert to grayscale

```
img1 = cv2.imread(image_1_with_path)
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
img2 = cv2.imread(image_2_with_path)
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
```

Detect SIFT features

```
sift = cv2.xfeatures2d.SIFT_create()           # create SIFT object  
keypoints_1, descriptors_1 = sift.detectAndCompute(img1, None)  
keypoints_2, descriptors_2 = sift.detectAndCompute(img2, None)
```

Match descriptors of two images

```
bf = cv2.BFMatcher(cv2.NORM_L1, crossCheck=True) # create feature matcher  
matches = bf.match(descriptors_1, descriptors_2) # match descriptors of both images  
matches = sorted(matches, key = lambda x:x.distance) # sort matches by distance
```

Draw first 50 matches

```
matched_img = cv2.drawMatches(img1, keypoints_1, img2, keypoints_2, matches[:50], img2, flags=2)
```

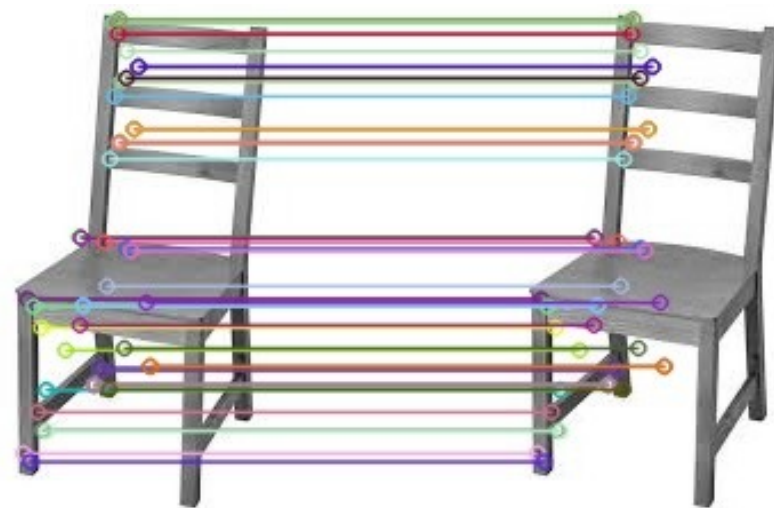
Save the image

```
cv2.imwrite(output_with_path, matched_img)
```

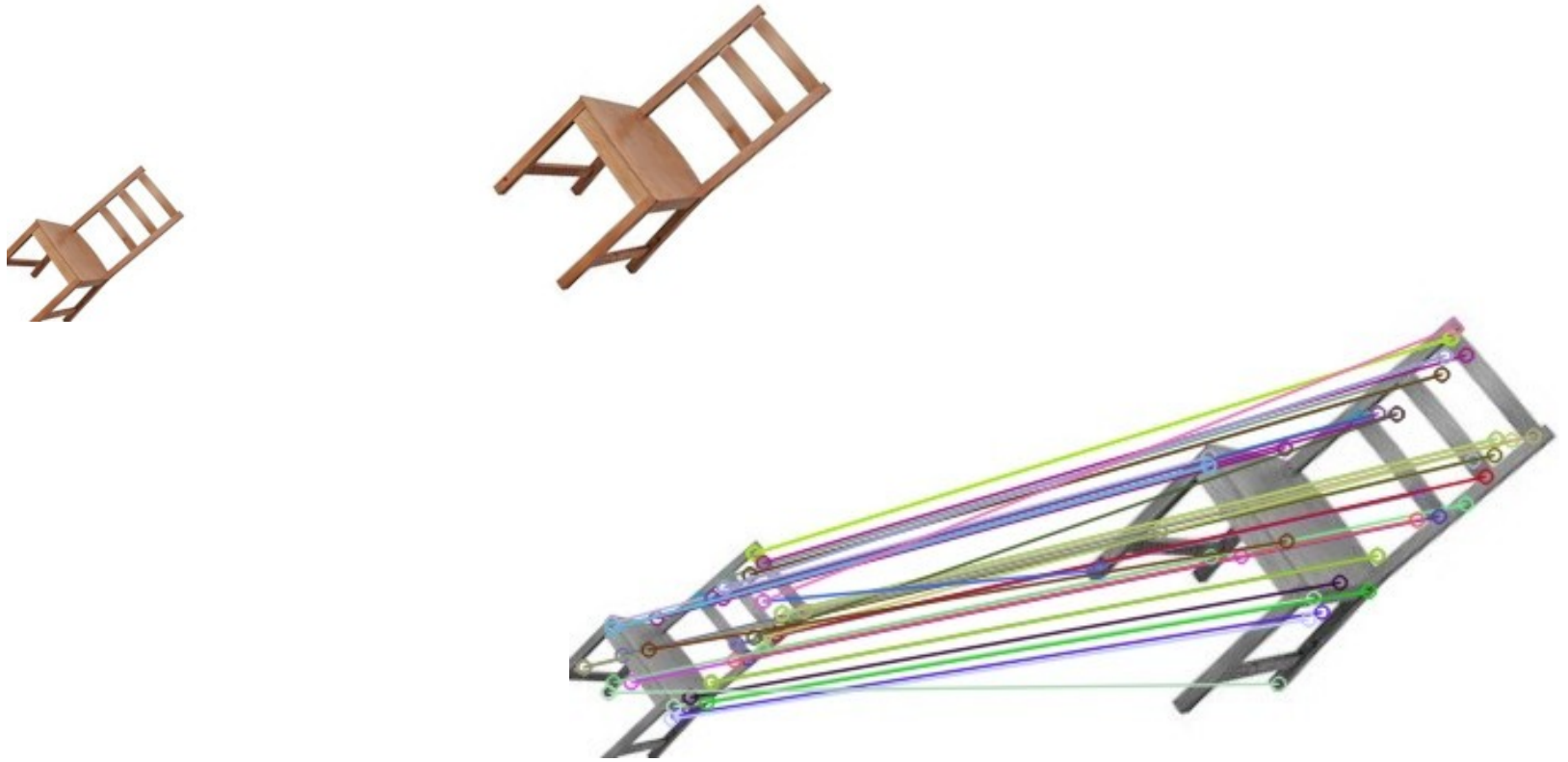
Show the image

```
cv2.imshow('image', matched_img)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

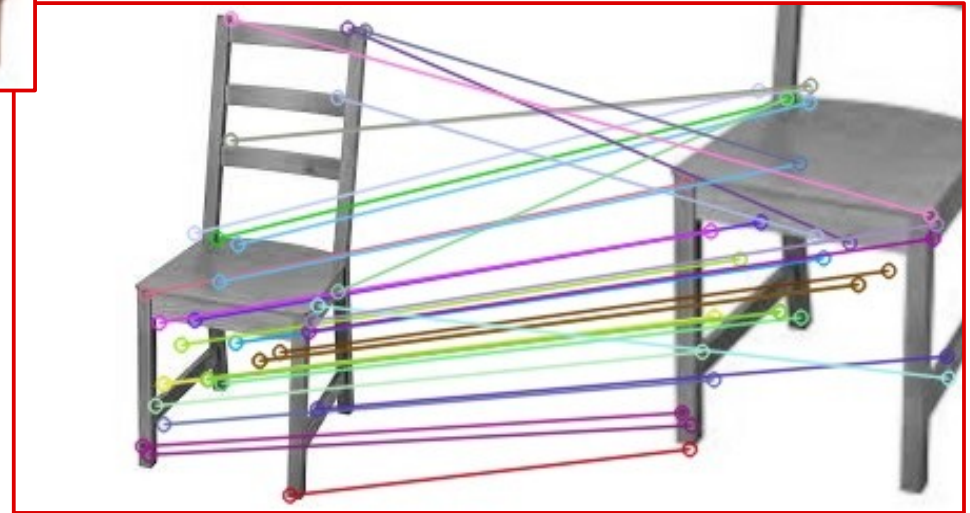
Same Size - Same Orientation – Same Illumination



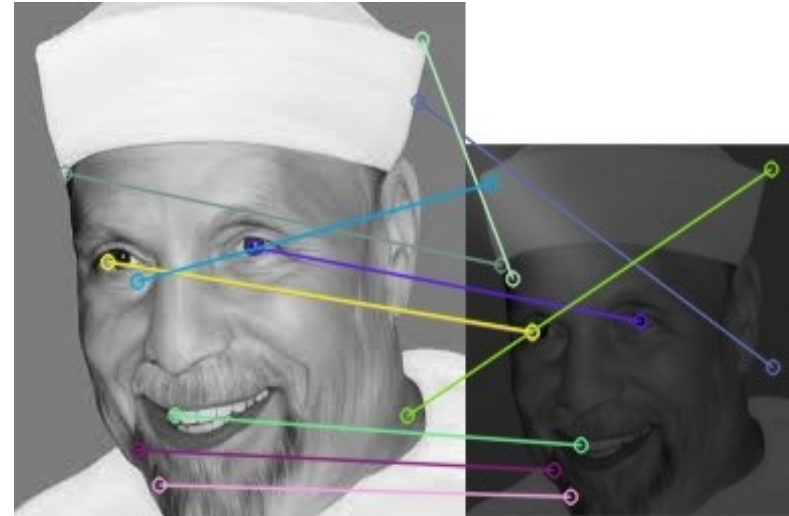
Different Sizes - Same Orientation – Same Illumination



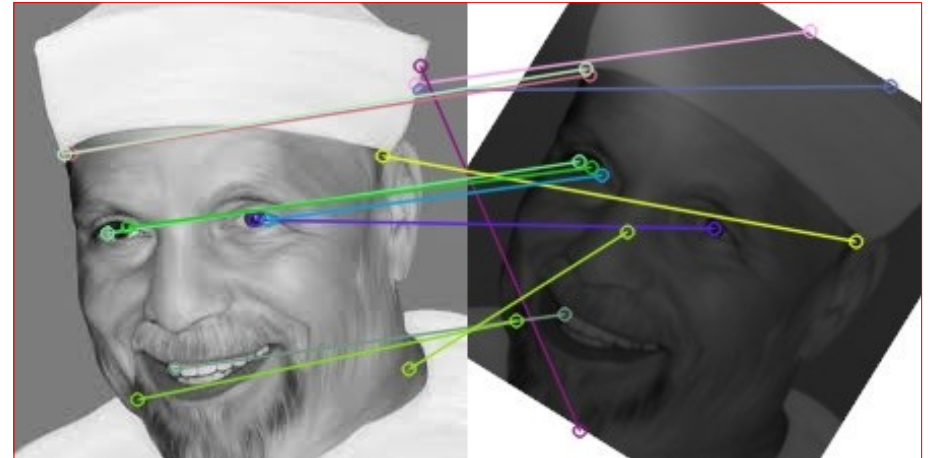
Different Sizes - Same Orientation – Same Illumination - Cropping



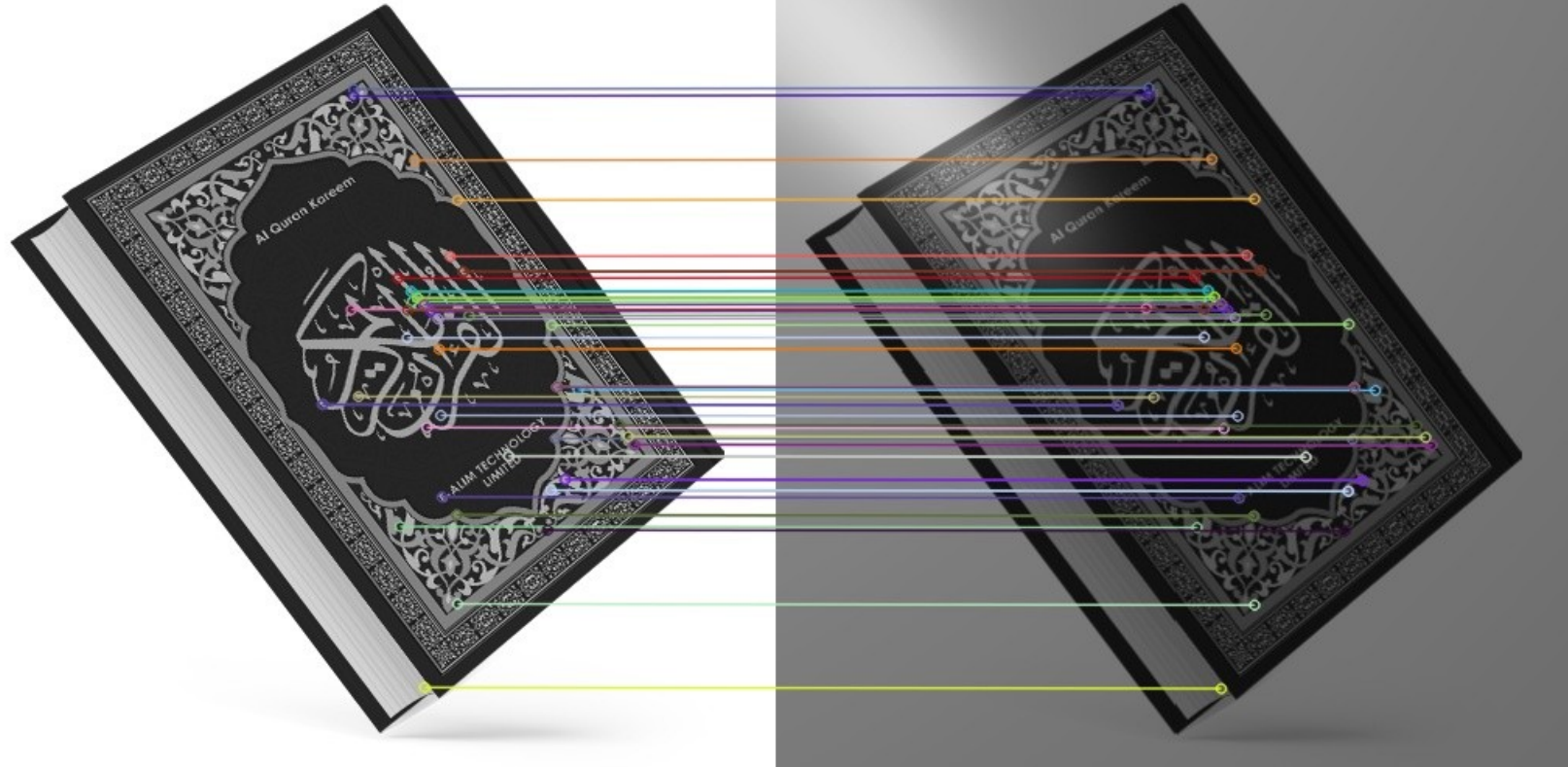
Different Sizes - Same Orientation – Different Illumination



Different Sizes - Same Orientation – Same Illumination - Cropping



Same Size - Same Orientation – Different Illumination



Different Sizes - Same Orientation – Same Illumination - Cropping



Different Sizes - Same Orientation – Same Illumination

